Level-4 Bounded Recursive Self-Improvement under a Published Definition: A Preregistered Test of a Self-Referential Improvement Mechanism

Junai P. Felix*
Independent Researcher, Taipei, Taiwan

October 2026
Version 1.3

Abstract

Claims of recursive self-improvement (RSI) are usually judged against criteria chosen by their authors. We instead test a self-modifying runtime module against a published capability definition: Level 4, *bounded recursive self-improvement*, in the taxonomy of Liu et al. (2026). Level 4 requires a system to propose, validate and apply modifications; to rewrite its own improvement mechanism, so that the mechanism becomes self-referential; and to sustain long-term progress within a bounded domain. In FELIX-C3, the improvement mechanism is an explicit, versioned object that contains the procedure for editing itself. The system proposes variants of the mechanism, validates each in a budget-neutral paired trial against the incumbent, and adopts it only by a margin that is itself part of the mechanism. We registered the definition, five definitional criteria, two causal criteria, the autonomy criteria, the seeds and two gates before writing any code; froze and hashed the build; and ran the confirmatory experiment once (24 previously unused seeds $\times$ 3 conditions, 32 rounds per mission, no operator). All definitional criteria passed. Every mission promoted audited revisions (24/24). Validated rewrites of the mechanism were adopted in 21 of 24 missions (60 of 166 trials), and in 17 of 24 an adopted rewrite also changed the rewriting procedure itself, although the trial that admits a rewrite cannot measure the effect of that part of it. Test solve rate rose from 27.4% to 40.1%, with second-half progress of +2.1 pp (18 seeds up, 2 down, 4 unchanged; exact one-sided Wilcoxon $p = 7.6 \times 10^{-6}$). All 72 missions reconciled with the frozen sources and ran with zero interventions under a supervisor that passed 14 of 14 fault-scenario groups and detected 8 of 8 deliberately broken builds. The causal criteria failed, as the power gate had predicted before the run: rewriting the mechanism did not significantly outperform the same system with its mechanism frozen (+0.35 pp, $p = 0.23$) or with rewrites adopted at random (+0.72 pp, $p = 0.079$). The system therefore meets the Level-4 definition as operationalised here, but we find no evidence that its self-referential rewriting is what drives its progress. Four further preregistered studies leave this unchanged: a transplant test (+0.49 pp, $p = 0.17$), two redesigns that stopped at their pilot gates, and a noise-aware redesign whose fresh-seed-validated rewrites did worse than the default (-1.12 pp). A companion preregistered study finds that the loop's policies repair 23% more real bugs from two external benchmarks than the starting policy, equally under every condition.

Keywords: recursive self-improvement · self-referential improvement · capability taxonomy · autonomous operation · preregistration · evaluation methodology

*Correspondence: junai.felixx@gmail.com.

Contents

1 Introduction **3**

2 Background and Related Work **4**

3 Target Definition and Its Operationalisation **5**
- 3.1 The definition, verbatim 5
- 3.2 From clauses to criteria 5
- 3.3 Verdict rules 6

4 Programme History: Three Stages **7**

5 System Under Test **9**
- 5.1 Disclosure boundary 9
- 5.2 Components and the improvement loop 9
- 5.3 The improvement mechanism 9
- 5.4 How `Imp` is rewritten 10
- 5.5 What is fixed 10

6 Experimental Design **11**
- 6.1 Task environment 11
- 6.2 Conditions 11
- 6.3 Mission protocol 11
- 6.4 Seeds 12

7 Preregistration, Gates and Analysis Plan **13**
- 7.1 What was registered, and when 13
- 7.2 Freeze 13
- 7.3 Gate 0: validity 13
- 7.4 Gate 1: pilot and power 14
- 7.5 Statistical procedures 14

8 Autonomy Evaluation Design **15**
- 8.1 Supervisor contract 15
- 8.2 Criteria 15
- 8.3 Fault-scenario suite 15
- 8.4 Mutation analysis 16

9 Results I: Development, Gates and Fault Suite **17**
- 9.1 Development 17
- 9.2 Gate 0 17
- 9.3 Gate 1 17
- 9.4 Fault suite on the frozen build 18

10 Results II: Definitional Criteria (Tier A) **19**
- 10.1 Sustained progress 19

11 Results III: Rewriting the Mechanism **21**
- 11.1 Frequency and timing 21
- 11.2 What was rewritten 21
- 11.3 Did the mechanism lower its own bar? 22
- 11.4 Did variants produce useful revisions? 22
- 11.5 What the trials could and could not validate 22

12 Results IV: Causal Criteria (Tier B) **24**
- 12.1 Tier B 24
- 12.2 Secondary analyses (reported only) 24
- 12.3 Improvement dynamics 25
- 12.4 Exploratory: rewriting and benefit 26

13 Results V: Autonomous Operation **27**

14 Follow-up Studies: Is the Recursion Productive? **28**
- 14.1 Transplant test 28
- 14.2 Is there headroom in the mechanism? 29
- 14.3 Redesign 1: validation within a mission 29
- 14.4 Redesign 2: validation across missions 30
- 14.5 Redesign 3: noise-aware validation across missions 32
- 14.6 Does the improvement transfer to real bugs? 33
- 14.7 Synthesis 34

15 Discussion **35**
- 15.1 What the result establishes 35
- 15.2 Why the recursion was not productive 35
- 15.3 Implications for Level-4 classification 35
- 15.4 Next steps 36

16 Limitations and Threats to Validity **37**

17 Integrity, Reproducibility and Disclosure **38**
- 17.1 Commitments 38
- 17.2 Determinism 38
- 17.3 Deviations and corrections 39
- 17.4 Attempt accounting 39
- 17.5 Author contributions and use of AI assistance 39
- 17.6 Audit package and reproduction 40

18 Conclusion **41**

References **42**

A Per-Seed Confirmatory Results **44**

B Development-Phase Disclosure **45**

C Statistical Details **46**

D Source Hashes **47**

1 Introduction

A system exhibits recursive self-improvement (RSI) when it improves the procedure by which it improves itself [1, 2]. Empirical systems now revise their own scaffolds, agent designs and code [3–8], and the word "recursive" is applied to many of them. What has been missing is an external yardstick. When the authors of a system also choose the criterion by which it counts as recursive, a positive result says little.

Two recent surveys supply such yardsticks. Liu et al. [9] grade self-improving agents on five levels. At Level 3 a system proposes, applies and validates modifications to its operational components while its *improvement mechanism* stays fixed. At Level 4, bounded recursive self-improvement, the system also rewrites that mechanism, which makes it self-referential, and the overall process sustains long-term progress within a bounded domain. Chen et al. [10] separate bounded self-refinement against fixed external evaluators from open-ended RSI. Both treat rewriting the improvement mechanism as the step that separates recursion from ordinary self-modification.

This paper tests one system against Liu et al.'s Level-4 definition. It reports the third stage of a programme whose first two stages passed preregistered tests of an internal depth criterion but, on inspection, kept the improvement mechanism fixed and so sit at Level 3 under the published taxonomy (Section 4). The third stage, FELIX-C3, makes the mechanism an explicit object that the system rewrites by a procedure that is itself part of the mechanism. We translated each clause of the definition into a criterion, added two causal criteria that the definition does not require, registered everything before writing code, and ran the confirmatory experiment once.

Contributions.

- An operationalisation of a published Level-4 RSI definition as five preregistered criteria, plus two causal criteria that ask whether rewriting the mechanism helps (Section 3).
- A complete record of the test process, from registration through development, freeze, gates, fault-suite validation and a single confirmatory run, including the deviations (Sections 7 and 17).
- A confirmatory result in which every definitional criterion and every autonomy criterion passes (Sections 10 and 13).
- A null causal result. The self-referential rewriting is present and validated, but it does not measurably beat a frozen or randomly rewritten mechanism (Section 12). We argue that Level-4 classification should require this counterfactual (Section 15).
- Four follow-up preregistered studies of whether the recursion is productive, a diagnosis of why none succeeds, and a summary of a companion study showing that the improvement loop's product transfers to real bugs (Section 14).

2 Background and Related Work

Recursive self-improvement. Good's argument that a machine able to improve its own design could trigger an "intelligence explosion" framed RSI as improvement of the improver [1]. Schmidhuber's Gödel machine made this precise as self-rewriting under a proof obligation that the rewrite increases expected utility [2], and Yampolskiy surveyed the conceptual limits of such software [11]. Learned optimisers [12] and evolved learning algorithms [13] showed that parts of the learning procedure can themselves be learned, although the outer procedure that learns them usually stays fixed.

Empirical self-improving agents. The Self-Taught Optimizer uses a language model to improve the scaffold that improves programs, including that scaffold itself [3]. Gödel Agent lets an agent inspect and modify its own logic at run time [4]. Automated design of agentic systems searches over agent code with a meta-agent [5]. The Darwin Gödel Machine maintains an archive of coding agents that rewrite their own code and validates each variant on benchmarks [6]. The Self-Improving Coding Agent edits its own implementation and reports large benchmark gains [7], and the Huxley-Gödel Machine guides self-modification by the estimated productivity of an agent's descendants [8]. Liu et al. place Gödel Agent, the Darwin Gödel Machine, the Self-Improving Coding Agent, the Huxley-Gödel Machine and several others at Level 4, and no system at Level 5 [9].

Capability taxonomies. Level schemes make capability claims comparable: levels of human interaction with automation [14], of driving automation [15], of general AI [16] and of agent autonomy [17]. Liu et al. grade self-improvement by which roles the system itself fills: proposer, modifier, validator, modification of the improvement mechanism, and generality across domains [9]. Chen et al. argue that the strength of self-improvement tracks the robustness of the evaluator, and that self-confirming loops appear when the system grades itself [10]. Both inform our design: the validator that gates every policy change is external and fixed, while the mechanism that generates and screens changes is the object under test.

Autonomous operation. We evaluate operation without a human present as *bounded Level-4 autonomy*, by analogy with high driving automation inside a defined operational domain [15]. The supervisor follows crash-only design principles [18, 19], and the fault suite is itself checked by mutation analysis [20, 21]: a suite that cannot fail is not evidence.

Preregistration and reporting. Preregistration separates confirmatory from exploratory analysis [22]. Machine-learning results are sensitive to seeds and to undisclosed analysis choices [23, 24]. We fix seeds, hold-out, conditions, statistics and pass rules before writing code, use exact nonparametric tests [25], and report bootstrap intervals [26] and Hodges–Lehmann estimates [27] as descriptive statistics only.

Program repair as a test domain. The task domain is a finite-grammar program-repair environment. Automated program repair has a long literature [28, 29]. Here it serves only as a bounded, cheaply verifiable domain in which every candidate repair can be checked exactly, so the external validator cannot be gamed.

3 Target Definition and Its Operationalisation

3.1 The definition, verbatim

Liu et al. define the levels in Section 2.2 of their survey [9]. The Level-4 definition reads in full:

> *"(L4) Bounded recursive self-improvement. The agent system can not only propose candidate modifications, validate their effects, and apply the validated modifications, but also rewrite its own improvement mechanism, which makes the improvement mechanism self-referential. Individual steps need not modify every component, but the overall process supports sustained long-term progress within a bounded domain."*

The improvement mechanism is defined in their Section 2.1.1:

> *"The improvement mechanism Imp governs how evidence and feedback are used to modify the agent system over time."*

At Level 3, by contrast, "the system controls the execution of improvement, but does not yet modify the mechanism that governs subsequent improvement cycles." Level 5 keeps the autonomy, self-reference and long-term progress of Level 4 and adds transfer of the improvement capability across broad and evolving domains. Their Table 1 grades systems on five columns: Proposer, Modifier and Validator (the programmatic improvement loop), Imp Modification (recursion), and Generality (domain).

3.2 From clauses to criteria

The survey classifies systems by their architecture; it does not prescribe a test. We therefore wrote one criterion per clause (Table 1) and fixed the thresholds before any code existed. Three interpretive choices deserve to be stated plainly.

(i) *"Rewrite its own improvement mechanism"* is read as the adoption of at least one validated change to an explicit Imp specification. An edit trialled and rejected does not count.

(ii) *"Self-referential"* is read structurally: the procedure that rewrites Imp must itself be part of Imp and editable by that same procedure. We also report how often an adopted edit actually changed the rewriting procedure.

(iii) *"Sustained long-term progress"* is read as statistically significant improvement on fresh test tasks across the *second* half of long missions, so that a system which improves early and then stalls fails.

Table 1: Operationalisation of the Level-4 definition (Tier A) and the added causal criteria (Tier B), as registered before implementation. All Tier A criteria refer to the live condition C6; $N = 24$ missions.

ID	Clause or question	Preregistered criterion
A-1	"propose candidate modifications, validate their effects, and apply the validated modifications"	At least one externally audited policy revision is promoted in ≥ 20 of 24 missions.
A-2	"rewrite its own improvement mechanism"	At least one validated Imp edit is adopted in ≥ 20 of 24 missions.
A-3	"which makes the improvement mechanism self-referential"	The Imp-editing procedure lies inside the editable Imp (verified in the frozen source). Reported: missions in which an adopted edit changed that procedure.
A-4	"sustained long-term progress within a bounded domain"	Test solve rate at round 32 minus round 16: mean $\geq +0.01$ and one-sided exact Wilcoxon $p < 0.01$.
A-5	Integrity	Sources match the freeze; every mission reconciles, including its Imp-version hash chain.
B-1	Does rewriting Imp help?	Round-32 solve, C6 $-$ C6f: mean $\geq +0.01$, $p < 0.01$.
B-2	Does evidence matter at the Imp level?	Round-32 solve, C6 $-$ C6r: mean $\geq +0.01$, $p < 0.05$.
H	Headroom (validity)	C6f final mean ≤ 0.85, otherwise the study is invalid.

3.3 Verdict rules

The verdict "Level 4 (definitional)" requires A-1 to A-5 and headroom. The stronger verdict "Level 4 with causal support" additionally requires B-1 and B-2. Both verdicts were to be reported whatever the outcome. Tier B goes beyond the definition on purpose. A system can satisfy every clause of the definition while its self-rewriting contributes nothing, and only a frozen-mechanism counterfactual can tell the two apart.

4 Programme History: Three Stages

The study reported here is the third stage of a programme. Each stage had its own protocol, registered before its code, its own freeze, gates and single confirmatory run, and its own previously unused seeds. Table 2 summarises the three. Absolute solve rates are not comparable across stages, because test sets and configurations differ.

Table 2: The three stages. Each row is a separate preregistered study with 24 confirmatory seeds and all missions run under the autonomous supervisor.

	Stage 1	Stage 2	Stage 3 (FELIX-C3)
What the system may rewrite	task policy; the generator that defines what revisions can express, selecting from a designer-enumerated space	as Stage 1, and the generator may extend its own eligible space	as Stage 2, and the improvement mechanism Imp, including the procedure that edits Imp
Rule that turns evidence into modifications	fixed	fixed	rewritable (Imp)
Primary contrast	writable vs. fixed generator	self-extending generator vs. the designer's full vocabulary	rewritable vs. frozen vs. randomly rewritten Imp
Primary result	+5.2 pp, 24/24 seeds, $p = 6.0 \times 10^{-8}$	+4.1 pp, 22/24 seeds, $p = 4.8 \times 10^{-7}$	Tier A pass; B-1 +0.35 pp, $p = 0.23$
Bound on the claim	a fixed generator pre-loaded with its whole eligible space did better (-2.0 pp)	a designer supplying the complete raw closure did better (-2.3 pp); no significant held-out gain	rewriting Imp not shown to help
Autonomy	0 interventions; 14/14 groups; 8/8 mutants	0 interventions; 14/14; 8/8	0 interventions; 14/14; 8/8
Level under Liu et al.	3 (borderline)	3 (borderline)	4 (definitional)

Why Stages 1 and 2 are Level 3. Both stages passed their own criteria. The criterion was a depth criterion: the number of writable layers above the task policy, with the top layer being a generator that shapes what policy revisions can express. Mapped onto Liu et al.'s taxonomy, the decisive question is whether the procedure that turns evidence into modifications can change. In Stages 1 and 2 it cannot. The rule that selects generator revisions, the operators that revise the policy, and the screening and integration rules are all fixed by the designer; only the representation and some adaptive statistics change. Liu et al. place a system whose "improvement mechanism remains fixed or externally maintained" at Level 3. The writable generator changes what modifications can be expressed, which is why we call the classification borderline, but the conservative reading is Level 3.

What Stage 3 changes. FELIX-C3 keeps the Stage 2 system and adds one thing: the improvement mechanism becomes an explicit, versioned object that the system rewrites, using a procedure that is part of

the object. The task environment, the supervisor and the statistics code are carried over byte-for-byte from earlier stages (Appendix D); the mission loop gained the trial machinery, and the revision and generation operators were extended so that their settings are read from Imp.

Table 3: Capability grading in the format of Liu et al.'s Table 1. A tick means the system performs the role itself, without a human in the loop.

	Proposer	Modifier	Validator	Imp modification	Generality	Level
Stage 1	✓	✓	✓	✗	–	3
Stage 2	✓	✓	✓	✗	–	3
Stage 3 (FELIX-C3)	✓	✓	✓	✓	–	4

The validator in all three stages is an external audit gate that the system cannot modify. In Liu et al.'s terms the system performs validation by invoking that gate on its own candidates; the gate's statistic and threshold are part of the fixed, bounded domain.

5 System Under Test

5.1 Disclosure boundary

We describe FELIX-C3 at the level of roles, interfaces and parameter groups. We withhold the representation of the task policy, the form of vocabulary elements, how revisions and vocabulary additions are constructed, the evidence format, and the internal form of each Imp parameter. Everything needed to check the experiment against the definition is reported: what Imp contains, how it is rewritten and validated, what stays fixed, the conditions, budgets, seeds, statistics and every result.

5.2 Components and the improvement loop

Figure 1 maps the system onto Liu et al.'s four stages: Diagnose, Propose, Evaluate and Integrate. In each round the system diagnoses failures on a batch of training tasks and records evidence. Under the current Imp, it proposes vocabulary additions and a fixed number of candidate policy revisions. It screens the candidates internally on the round's selection batch. If the best candidate solves more selection tasks than the incumbent policy, it is sent to an external audit on a separate, larger batch, and it is integrated (promoted) only if the audit shows a gain of at least 2 points with a one-sided sign-test $p \leq 0.10$.

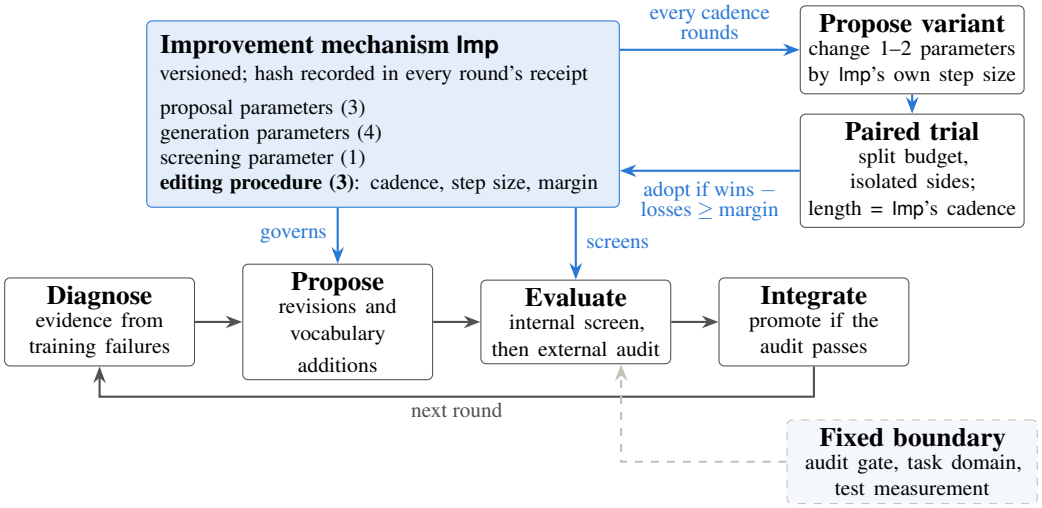


Figure 1: FELIX-C3 in Liu et al.'s Diagnose–Propose–Evaluate–Integrate frame. The lower row is the improvement loop that runs every round. Imp (upper left) governs how proposals are generated and screened. The loop on the right rewrites Imp: a variant is proposed, validated in a paired trial against the incumbent, and adopted only by a margin. The step size, the trial cadence and the adoption margin are themselves Imp parameters, so the rewriting procedure is rewritten by itself. The audit gate, task domain and test measurement are fixed and cannot be modified by the system.

5.3 The improvement mechanism

Imp is a single structured object with 11 parameters in four groups.

- *Proposal* (3 parameters) govern how evidence becomes candidate policy revisions: which kinds of revision are favoured, how strongly they are regularised, and how large they are.
- *Generation* (4 parameters) govern how the vocabulary of possible revisions grows: how many additions are made per round, how widely the space is sampled, and which construction forms are preferred.
- *Screening* (1 parameter) governs which internal candidate is sent to the external audit.

- *Editing procedure* (3 parameters) govern the rewriting of Imp itself: how often a variant is trialled (the trial cadence, which is also the trial length in rounds), how far a variant moves from the incumbent (the step size), and how decisively it must win to be adopted (the adoption margin).

Every version of Imp is hashed, and the hash after each round is written into that round's receipt, so the full version history of Imp is part of the mission's tamper-evident chain. All three conditions start from the same initial Imp.

5.4 How Imp is rewritten

Propose. When the round number is a multiple of the incumbent's trial cadence and a full trial still fits before the end of the mission, the system proposes a variant. It changes one or two parameters, chosen at random, by an amount scaled by the incumbent's step size. Any parameter can be chosen, including the three that define the editing procedure.

Validate. For the next trial window (as many rounds as the cadence), the candidate budget of each round is split equally. Half the policy revisions are proposed under the incumbent Imp and half under the variant. Each side also generates its own vocabulary additions, and neither side can see or use the other's. Both sides are scored on the same selection tasks, and the round is a win, loss or tie for the variant according to the mean solve count of each side's best two candidates. Policy promotion follows the usual rule during a trial: the best of the eight candidates, from either side, is screened with the incumbent's screening parameter and, if it beats the incumbent policy, audited. The total budget per round is therefore identical with or without a trial.

What a trial can measure. A trial measures only the parameters that act on candidate generation: the proposal and generation groups. The variant's screening parameter and its three editing-procedure parameters are not exercised during its own trial, because screening uses the incumbent's value and the trial runs on the incumbent's cadence and margin. Their effects appear only after adoption, in later rounds and later trials. Section 11.5 quantifies the consequences.

Apply. At the end of the window the variant is adopted if its wins minus losses reach the incumbent's adoption margin. If it is rejected, the vocabulary additions it produced are removed, except any that the current policy already uses.

Why this is self-referential. The step size that shapes a variant, the cadence that sets when and how long it is tested, and the margin that decides whether it is adopted are all read from the current Imp. An adopted variant that changes any of them changes how the next variant is proposed, tested or judged. The mechanism that rewrites Imp is therefore inside Imp and is rewritten by itself. The first adoption in any mission is always judged under the initial margin.

5.5 What is fixed

The bounded domain consists of the task families, the external audit gate (its batch size, gain threshold and significance threshold), the per-round candidate budget, the number of rounds and the test measurement. None of these can be changed by the system, and none is read from Imp. Test checkpoints never feed back into the system.

6 Experimental Design

6.1 Task environment

The domain is finite-grammar program repair. Each task is a short program with one to three injected faults, a set of visible input–output examples, and a set of held-back check examples. A task is solved if the repair found within a fixed evaluation budget passes both the visible and the held-back examples. Tasks come from 16 parametric families. Twelve are development families used for training, selection and audit. Four are held out from all training and appear only in the test measurement: `any_neg`, `count_gt`, `modwrap` and `poly`. They were derived deterministically from the freeze hash, so they could not be chosen after seeing results.

6.2 Conditions

The three conditions differ only in how Imp may change (Table 4). They share the initial Imp, the per-round candidate budget, the number of rounds and the task stream for each seed.

Table 4: Conditions. All other settings are identical.

Condition	Imp	Role
C6	rewritable, evidence-validated	The live system. Tier A is tested on this condition.
C6f	frozen at the initial Imp	The Level-3 counterfactual: the identical system without Imp rewriting.
C6r	rewritable on the same schedule; adoption by coin flip	Evidence-blind control: identical trials and budget split, but the trial outcome is ignored.

6.3 Mission protocol

A mission is one condition run on one seed for 32 rounds. In each round the system proposes 8 candidate policy revisions, screens them on a selection batch of 96 training tasks, and, if the best one beats the incumbent on that batch, audits it against the incumbent on 300 fresh training tasks. The audit promotes the candidate if the gain is at least 0.02 and the one-sided sign test gives $p \leq 0.10$. Each task attempt is capped at 12 evaluations.

Test checkpoints. At rounds 0, 16 and 32 the current policy is evaluated on the mission's test set: 320 tasks, 20 from each of the 16 families, generated from a task stream separate from selection and audit. No test task is identical to any training task; in the development families, many share program text with some training task but carry different examples (Section 16). The same test set is used at all three checkpoints of a seed, so progress between checkpoints is a paired measurement. Checkpoint results are written to the mission state but are never read by the system.

Table 5: Frozen configuration.

Parameter	Value	Parameter	Value
Rounds per mission	32	Audit tasks	300
Candidates per round	8	Audit gain threshold	0.02
Selection tasks per round	96	Audit sign-test p (max)	0.10
Evaluation budget per task	12	Test tasks per seed	320 (20×16)
Initial trial cadence	4 rounds	Checkpoints	0, 16, 32
Initial adoption margin	1	Families (dev / held out)	12 / 4

6.4 Seeds

Seed ranges were fixed in the protocol. Confirmatory seeds had never been used by any earlier stage or development run.

Table 6: Seed allocation.

Use	Seeds	Families
Development	1300–1399	all 16 (hold-out not yet determined)
Gate 0	1401–1408	all 16, trained and tested
Gate 1 pilot	3201–3208	12 development families only
Confirmatory	7101–7124	all 16 (12 trained, 4 held out)

7 Preregistration, Gates and Analysis Plan

7.1 What was registered, and when

The protocol was posted to a hosted document store that assigns server-side timestamps, before any code for this stage existed. It contains: the verbatim target definition; the conditions; the seed ranges; the Tier A and Tier B criteria with thresholds; the autonomy criteria; the headroom rule; both gates and their decision rules; the list of permitted and forbidden actions; and a commitment to report NO-GO and FAIL outcomes in full. Three further records were posted at fixed points: a freeze record (source hashes, configuration, initial lmp, held-out families, development disclosures and known weaknesses), a gate record before the confirmatory run, and a results record after it. Figure 2 shows the chronology.

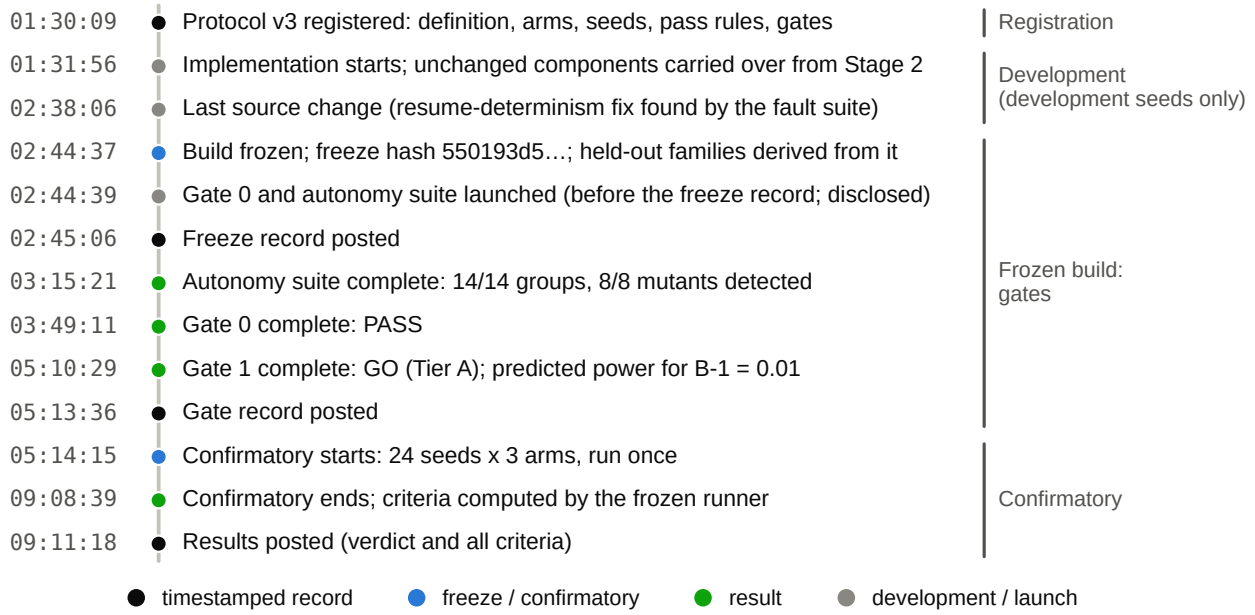


Figure 2: Study chronology on the single study day (UTC). Black: records posted with server-side timestamps. Blue: freeze and confirmatory. Green: results. Grey: development and the launch of the first gate. Implementation began after the protocol was registered, and the last source change preceded the freeze. The first gate was launched two seconds after the freeze and 27 seconds before the freeze record was posted; no result existed until 03:15 (Section 17.3).

7.2 Freeze

After development, all eight source files, the configuration and the initial lmp were hashed into a single freeze hash, `550193d5...b3801`. The held-out families were then derived from that hash. Before every subsequent stage, and again after the confirmatory run, the sources were re-verified against the freeze. After the freeze, no source change, rule change, seed change, seed exclusion or rerun of the confirmatory experiment was permitted.

7.3 Gate 0: validity

Gate 0 runs C6 and C6f on eight development seeds (1401–1408) and passes only if all three checks hold:

(i) *determinism*: rerunning C6 on seed 1401 from scratch reproduces the receipt chain and the full lmp version history;

(ii) *distinct trajectories*: at least 6 of the 8 C6 missions follow distinct trajectories, so that seeds are not collapsing onto a few outcomes;

(iii) *headroom*: the C6f mean final solve rate is at most 0.85.

7.4 Gate 1: pilot and power

Gate 1 runs all three conditions on eight pilot seeds (3201–3208), training and testing on the 12 development families only. It returns GO for the confirmatory experiment if a validated Imp edit is adopted in at least 6 of 8 C6 missions *and* the mean second-half progress of C6 is at least 0.01; otherwise NO-GO, and the study stops and is reported. Gate 1 also estimates the power of A-4 and B-1 at $N = 24$ by bootstrap resampling of the eight pilot differences (10,000 resamples, fixed seed). A low power for Tier B predicts that Tier B will fail but does not stop the confirmatory run; it is recorded so that the prediction is on file before the result.

7.5 Statistical procedures

All confirmatory tests are one-sided exact Wilcoxon signed-rank tests on the 24 per-seed paired differences [25]. Zero differences are dropped, tied absolute differences receive midranks, and the null distribution of the signed-rank statistic is computed exactly, by dynamic programming over the observed ranks, so the test is exact under ties. The implementation was checked against a reference exact test on tie-free data and against brute-force enumeration with ties. Each criterion pairs its test with a minimum mean effect, so a significant but negligible difference cannot pass. There are seven preregistered criteria, each with its own threshold and no correction for multiplicity; the verdict rules require every Tier A criterion jointly. Bootstrap 95% intervals (percentile, 20,000 resamples) and Hodges–Lehmann estimates [26, 27] are descriptive only. All analyses labelled "reported" or "exploratory" were not preregistered as criteria.

8 Autonomy Evaluation Design

8.1 Supervisor contract

Every mission in every stage of the study runs under an autonomous supervisor. The supervisor holds an exclusive lease on the mission, starts the worker process, watches a heartbeat, kills and restarts a hung worker, restarts a crashed worker at most twice, restores the last good checkpoint if the current one is corrupt, reaps orphaned workers left by a dead supervisor, honours cancellation, and verifies the receipt chain before it reports completion. State is written atomically with checksums, and each round appends a receipt that includes the hash of the previous receipt. A mission therefore ends in exactly one of a small set of terminal states (completed, stopped for budget, stopped for plateau, cancelled, or stopped at the restart limit), each with a recorded reason, and never in an undefined state.

8.2 Criteria

Table 7: Preregistered autonomy criteria (bounded Level-4 autonomy).

ID	Criterion	Threshold
Auto-1	No human intervention in any confirmatory mission	0 interventions
Auto-2	Fault-scenario suite on the frozen build	≥ 10 groups, all pass
Auto-3	Mutation analysis: deliberately broken supervisor builds detected by the suite	≥ 4 detected

8.3 Fault-scenario suite

The suite contains 14 groups (Table 8). Each group injects a fault into a short C6 mission (6 rounds) and checks the outcome against a fault-free reference run. For this stage the groups were configured so that Imp trials start early and run in short windows. Crashes, hangs, reopenings and checkpoint corruption therefore strike in the middle of an Imp trial, which is the hardest case for exact resumption: a resumed mission must reproduce both the policy history and the Imp version history of the uninterrupted run.

Table 8: Fault-scenario groups and what each must show. "Matches reference" means the final state, receipt chain and Imp history equal those of the fault-free run.

Group	Fault	Required outcome
G1	none	completes; matches reference; no restarts, no interventions; receipts verify
G2	transient worker crash	exactly one restart; completes; matches reference
G3	crash after a receipt, before the checkpoint	one restart; the uncommitted receipt is dropped; matches reference
G4	persistent crash	exactly two restarts (three worker starts); stops at the restart limit; no progress past the fault
G5	worker hang	one watchdog kill; completes; matches reference
G6	operator cancellation	worker terminated; stops as cancelled; the cancellation is recorded; no result written
G7	second supervisor on the same mission	second supervisor rejected; only the first spawns workers; matches reference
G8	cancel, then reopen	cancellation holds; reopening a stopped mission spawns nothing and spends nothing; a reopened mission completes with its spent budget carried over
G9	out-of-range score	flagged invalid and never selected; completes
G10	evaluation budget exhausted	stops at a round boundary with reason recorded; no result written
G11	corrupted checkpoint	restores the previous checkpoint and completes, matching reference; an unrecoverable state stops without spawning a worker
G12	no promotions (plateau)	stops after the plateau limit; status recorded; no result written
G13	supervisor killed mid-mission	orphaned worker reaped; a new supervisor takes the lease; completes; matches reference
G14	receipt chain tampered	tampering detected

8.4 Mutation analysis

A fault suite that passes proves little unless it can fail. We built eight mutant supervisors, each with one safety mechanism removed or broken: no retry after a crash; one retry too many; no lease (two supervisors may run the same mission); cancellation ignored; no watchdog for hangs; no restoration of the previous checkpoint; no reaping of orphaned workers; and no verification of the receipt chain. A mutant counts as detected if at least one group fails under it.

9 Results I: Development, Gates and Fault Suite

9.1 Development

Development used seeds 1301–1305, all 16 families, and was disclosed in full in the freeze record before any gate was run (Appendix B). The first design ran 40 rounds and let variant candidates use vocabulary added under the incumbent. Imp edits were rarely adopted (0 of 9 and 1 of 9 trials in the two missions), and C6 finished below C6f on average (-1.0 pp over the two missions). A probe of fixed alternative settings of Imp showed that the mechanism had some headroom: one alternative setting reached 41.2% and 41.9% against 41.2% and 37.5% for the default, while another did not help. Four changes followed: the two sides of a trial were isolated from each other's additions; the trial statistic was made finer (mean of the best two candidates rather than the best one); and, to fit the compute budget, the mission was shortened to 32 rounds and one generation parameter of the initial Imp (a sampling budget) was reduced. In the final development check both missions adopted two edits; in one of them both adoptions changed the editing procedure.

A defect caught by the fault suite. Running the suite on the development build, 7 of 14 groups failed: missions resumed after a fault diverged from uninterrupted ones. The cause was that one random draw depended on the iteration order of a mapping inside Imp, and that order changed when Imp was reloaded from a checkpoint. The draw was made order-independent, after which all 14 groups passed. This was the last source change before the freeze. It is direct evidence that the suite detects real defects, not only planted ones.

9.2 Gate 0

Table 9: Gate 0 (seeds 1401–1408). Result posted before the confirmatory run.

Check	Result	Required	
Determinism (C6, seed 1401, rerun from scratch)	receipts and Imp history identical	identical	**PASS**
Distinct C6 trajectories	8 of 8	≥ 6	**PASS**
C6f mean final solve rate	0.418	≤ 0.85	**PASS**

On these eight seeds C6 finished below C6f on six (means 0.406 and 0.418). This was on file before the confirmatory run.

9.3 Gate 1

Gate 1 returned GO (Table 10, Figure 3). A validated Imp edit was adopted in all eight pilot missions, and second-half progress was positive on all eight seeds (mean +2.2 pp), giving a bootstrap power of 1.00 for A-4 at $N = 24$. The Tier B prediction was negative. The mean pilot difference C6 $-$ C6f was +0.1 pp, and the power for B-1 was 0.01; C6 $-$ C6r was -0.3 pp. The gate record therefore stated, before the confirmatory run, that Tier B was expected to fail.

Table 10: Gate 1 pilot (seeds 3201–3208, development families). Test solve rate.

Seed	C6 round 0	C6 round 16	C6 round 32	C6f round 32	C6r round 32
3201	0.300	0.354	0.367	0.342	0.412
3202	0.329	0.392	0.400	0.425	0.429
3203	0.250	0.379	0.429	0.438	0.412
3204	0.250	0.408	0.429	0.417	0.412
3205	0.217	0.371	0.396	0.354	0.367
3206	0.246	0.383	0.404	0.421	0.396
3207	0.275	0.425	0.442	0.450	0.446
3208	0.263	0.358	0.379	0.392	0.392
Mean	0.266	0.384	0.406	0.405	0.408

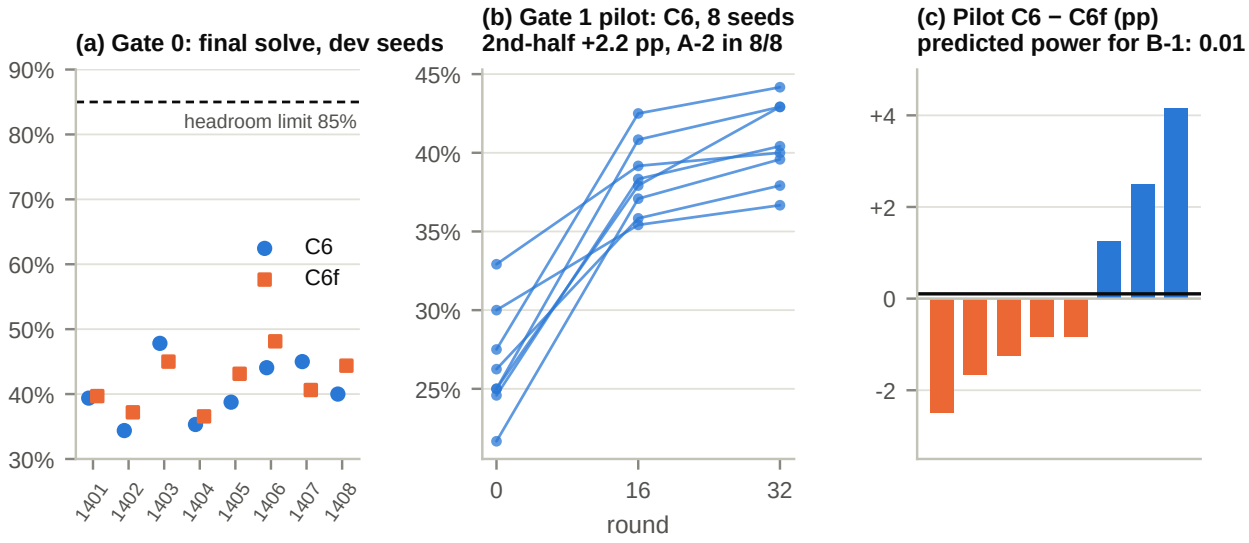


Figure 3: Gates. (a) Gate 0 final solve rates on development seeds; both conditions are far below the headroom limit. (b) Gate 1 pilot trajectories of C6; every seed improves in the second half. (c) Pilot differences C6 − C6f, sorted; the mean (black line) is near zero, which is why the predicted power for B-1 was 0.01.

9.4 Fault suite on the frozen build

The frozen build passed all 14 groups, and each of the eight mutant supervisors failed at least one group (Section 13, Figure 12). Auto-2 and Auto-3 therefore passed before the confirmatory run began.

10 Results II: Definitional Criteria (Tier A)

The confirmatory experiment ran once, on seeds 7101–7124, with all 72 missions completed. No seed was excluded. Table 11 gives the Tier A outcome.

Table 11: Tier A: the clauses of the Level-4 definition. Confirmatory seeds 7101–7124, condition C6.

ID	Result	Required	
A-1	24 of 24 missions promoted at least one audited revision (134 promotions; 5.6 per mission)	$\geq 20/24$	**PASS**
A-2	21 of 24 missions adopted at least one validated Imp edit (60 of 166 trials; 2.5 per mission)	$\geq 20/24$	**PASS**
A-3	Editing procedure inside the editable Imp: verified. In 17 of 24 missions an adopted edit changed the editing procedure (25 of the 60 adopted edits)	structural	**PASS**
A-4	Round 16 to round 32: +2.12 pp (95% CI [+1.33, +2.97]); 18 up, 2 down, 4 unchanged; $p = 7.6 \times 10^{-6}$	$\geq +1.0$ pp, $p < 0.01$	**PASS**
A-5	Sources match the freeze; 72 of 72 missions reconcile, including the Imp hash chain	all	**PASS**
H	C6f final mean 0.398	≤ 0.85	**PASS**

Verdict: Level 4 (definitional).

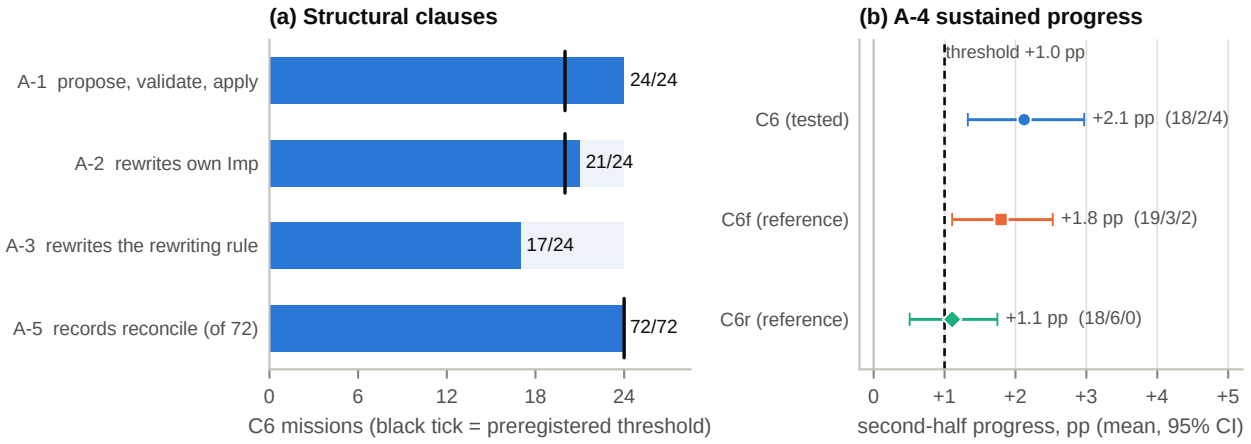


Figure 4: Tier A. (a) Missions meeting each structural clause, with the preregistered threshold. (b) Second-half progress (round 32 minus round 16) with bootstrap 95% intervals and seed counts (up/down/unchanged). Only C6 is tested; the other two conditions are shown for reference.

10.1 Sustained progress

All three conditions start from the same policy and the same Imp, so they share the round-0 value (27.4%). C6 reached 38.0% at round 16 and 40.1% at round 32 (Figure 5). Most of the gain comes in the first half (+10.6 pp, 24 of 24 seeds), and the second half adds a smaller, still significant +2.1 pp (Hodges–Lehmann estimate +1.9 pp). A-4 asks whether progress continues, not whether it accelerates, and it does continue: 18 of 24 seeds improve in the second half and only two decline.

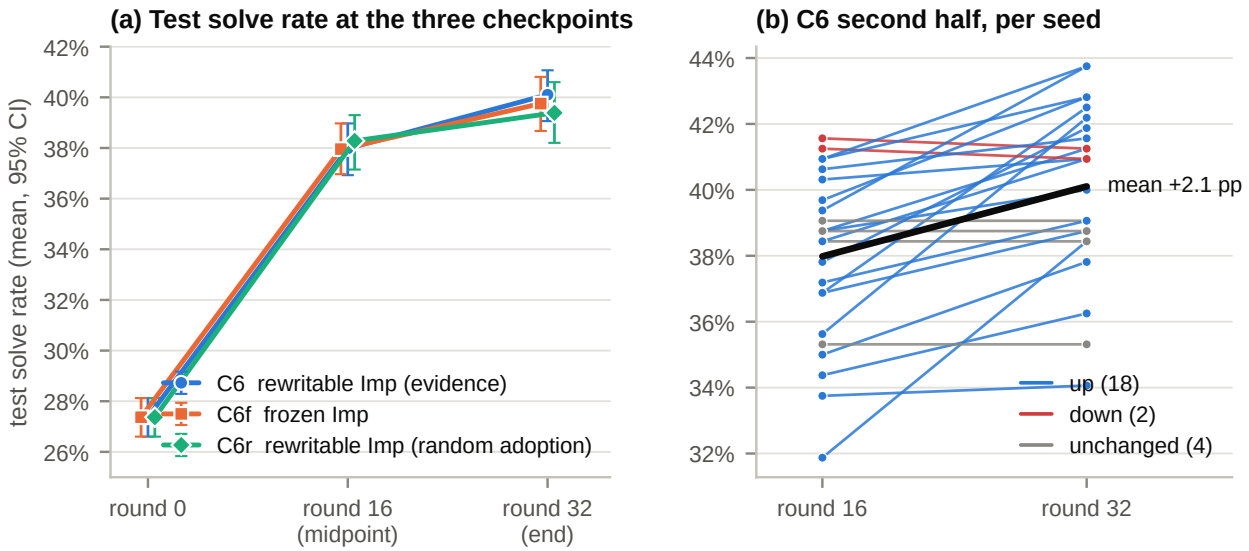


Figure 5: Test solve rate. (a) Means and bootstrap 95% intervals for the three conditions at rounds 0, 16 and 32, offset horizontally for legibility. The three conditions are indistinguishable at every checkpoint. (b) Each C6 seed from round 16 to round 32; black is the mean.

Table 12: Checkpoint means (test solve rate, %) and progress (pp) by condition. Progress values are means with one-sided exact Wilcoxon p against zero.

Condition	Round 0	Round 16	Round 32	First half	Second half
C6	27.4	38.0	40.1	+10.6 ($p = 6 \times 10^{-8}$)	+2.1 ($p = 7.6 \times 10^{-6}$)
C6f	27.4	38.0	39.8	+10.6 ($p = 6 \times 10^{-8}$)	+1.8 ($p = 1.9 \times 10^{-5}$)
C6r	27.4	38.3	39.4	+10.9 ($p = 6 \times 10^{-8}$)	+1.1 ($p = 1.6 \times 10^{-3}$)

Table 12 contains the result that matters most for interpretation. The frozen-mechanism control C6f also sustains progress in the second half (+1.8 pp, 19 up, 3 down). A-4 is satisfied by the system as a whole, as the definition asks, but the same property holds without lmp rewriting. Section 12 returns to this point.

11 Results III: Rewriting the Mechanism

11.1 Frequency and timing

Across the 24 C6 missions the system ran 166 Imp trials (6.9 per mission) and adopted 60 (36%). Three missions adopted nothing; the others reached between two and five versions of Imp (mean 3.5 versions per mission, counting the initial one). Rewriting was not confined to the start: 31 of the 60 adoptions occurred in the second half (Figure 6). Where an adopted edit changed the trial cadence, later trials in that mission ran on the new schedule, which is visible as irregular spacing in the raster (for example seeds 7106 and 7117).

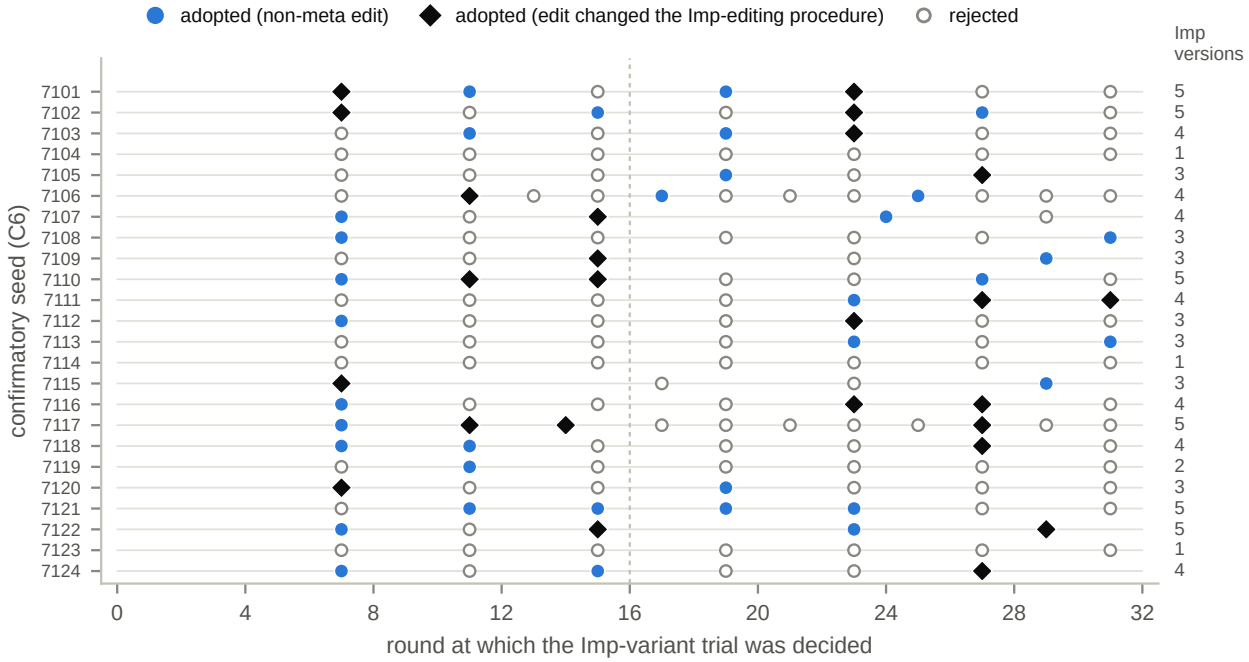


Figure 6: Every Imp trial in the confirmatory C6 missions, by the round in which it was decided. Blue: adopted. Black diamonds: adopted edits that changed the editing procedure (cadence, step size or margin). Hollow: rejected. Right: number of Imp versions the mission went through. The dotted line marks the midpoint checkpoint.

11.2 What was rewritten

Every parameter group was trialled and adopted at least once (Figure 7). Edits touching the screening parameter were adopted in 3 of 24 trials, against 9 of 21 under coin-flip adoption in C6r; because that parameter is not exercised during a trial (Section 11.5), we do not interpret the difference. Proposal and generation edits were adopted somewhat less often than under coin flip (37% and 35%, against 47% and 41%). Edits to the editing procedure itself were adopted at 41% (27 of 66 parameter touches), about the same rate as under coin flip (40%).

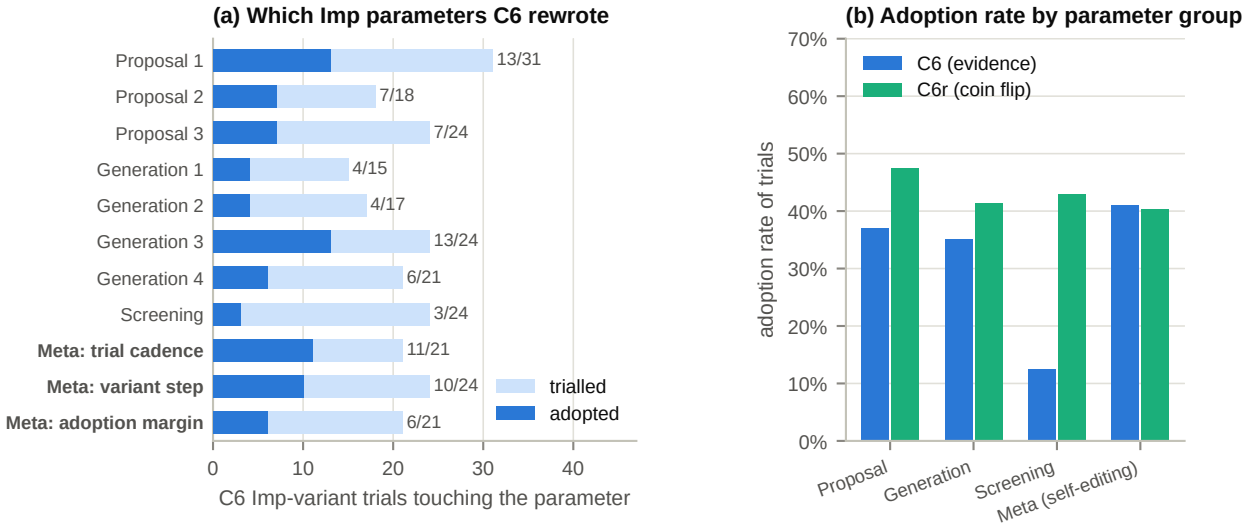


Figure 7: (a) Imp parameters touched by trials in C6 (light) and by adopted edits (dark); labels give adopted/trialled. Parameters are shown by group; their internal meaning is withheld. Bold: the three parameters of the editing procedure. A trial can touch two parameters. (b) Adoption rate per group under evidence-based adoption (C6) and coin-flip adoption (C6r).

11.3 Did the mechanism lower its own bar?

A self-referential mechanism can, in principle, make adoption easier for itself by reducing its adoption margin, which would inflate A-2. Three facts rule this out here. First, every one of the 21 first adoptions was judged under the initial margin, so A-2 does not depend on any self-modified margin. Second, among adopted edits the margin went up three times and down three times; it ended at its initial value in 20 of 24 missions, below it in two and above it in two. Third, the other two procedure parameters also moved in both directions (cadence: four increases, six decreases; step size: five and five).

11.4 Did variants produce useful revisions?

Trials were active in 643 of the 768 C6 rounds (84%). In those rounds, candidates proposed under the variant Imp were the ones sent to audit 253 times, against 243 for the incumbent, and they were promoted 50 times, against 49. Variants were therefore full participants in the improvement loop, not inert proposals. At the end, the final Imp differed from the initial one in 21 of 24 missions, by 3.3 of 11 parameters on average among those 21.

11.5 What the trials could and could not validate

Because a trial exercises only the proposal and generation parameters (Section 5), we separated adoptions by what they touched. Of the 166 C6 trials, 32 touched only parameters that a trial cannot exercise; 13 of these were adopted, all of them edits to the editing procedure. The remaining 47 adoptions touched at least one exercised parameter.

- *A-2 is robust to this.* Every one of the 21 missions that adopted an edit adopted at least one edit touching an exercised parameter, so A-2 holds under the stricter reading that only measurable edits count.
- *A-3 is structural, not validated.* Of the 25 adopted edits that changed the editing procedure, 12 arrived together with an exercised parameter and 13 were adopted on trials that could not reflect them. The procedure was rewritten by itself in 17 of 24 missions, as A-3 requires, but no trial measured whether a

given change to the procedure was beneficial. Doing so would need a second-order comparison across several later trials, which this design does not contain.

Liu et al.'s definition requires that the improvement mechanism be rewritten and self-referential; it does not specify how changes to the mechanism itself must be validated. We report the gap because it is exactly where a stronger Level-4 system would differ from this one.

12 Results IV: Causal Criteria (Tier B)

12.1 Tier B

Both causal criteria failed (Table 13, Figure 8), as the gate record had predicted. Rewriting lmp on evidence did not produce a significantly better final policy than leaving lmp frozen, nor than rewriting it on the same schedule with adoption decided by a coin flip.

Table 13: Tier B: causal criteria (round-32 test solve rate, paired by seed).

ID	Contrast	Mean (pp)	95% CI	Seeds +/ − /0	p
B-1	C6 − C6f: does rewriting lmp help?	+0.35	[−0.47, +1.15]	14/10/0	0.23
B-2	C6 − C6r: does evidence matter?	+0.72	[−0.14, +1.60]	15/8/1	0.079

Both **FAIL** (required: mean $\geq +1.0$ pp with $p < 0.01$ and $p < 0.05$ respectively).

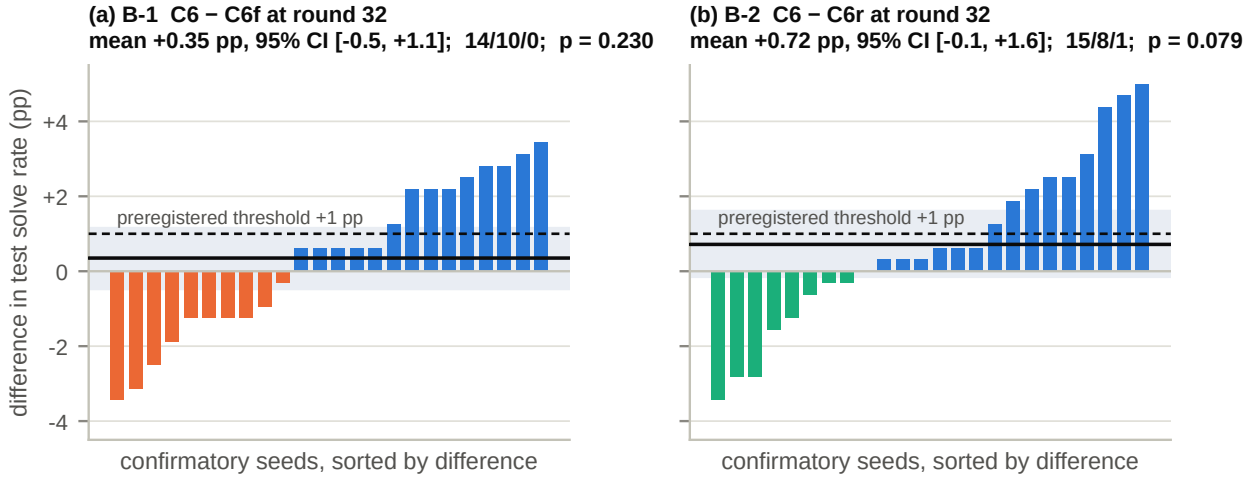


Figure 8: Per-seed paired differences at round 32, sorted. Black line: mean; shaded band: bootstrap 95% interval; dashed line: the preregistered minimum effect of +1 pp. Blue bars favour C6.

12.2 Secondary analyses (reported only)

Table 14: Secondary paired comparisons (one-sided exact Wilcoxon; no correction for multiplicity).

Comparison	Mean (pp)	95% CI	Seeds +/ − /0	p
C6 − C6f, round 16	+0.03	[−0.91, +0.94]	13/11/0	0.49
C6 − C6f, second-half progress	+0.33	[−0.66, +1.26]	12/10/2	0.21
C6 − C6f, held-out families	+0.89	[−1.30, +2.97]	13/8/3	0.22
C6 − C6f, development families	+0.17	[−0.50, +0.82]	12/8/4	0.24
C6 − C6r, held-out families	+0.31	[−1.98, +2.86]	9/10/5	0.59
C6 − C6r, development families	+0.85	[−0.03, +1.72]	16/7/1	0.040
C6r − C6f, round 32	−0.36	[−1.16, +0.43]	9/12/3	0.77

None of these comparisons was a criterion, and none survives any correction for the seven comparisons in the table. The one nominal result (C6 − C6r on development families, $p = 0.040$) should be read as a hypothesis

for a future preregistered test, not as evidence. Held-out and development families behave alike (Figure 9). Per-family differences between C6 and C6f range from −1.9 to +5.0 pp, without a pattern that separates held-out from development families.

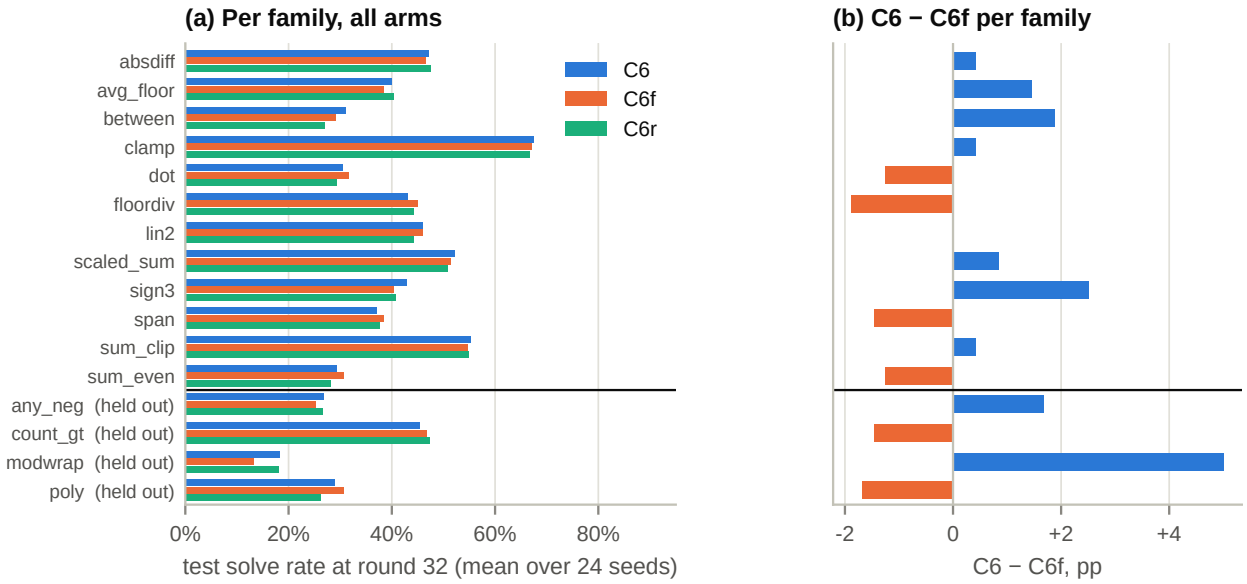


Figure 9: Per-family test solve rate at round 32. (a) All conditions; held-out families below the black line. (b) C6 − C6f per family.

12.3 Improvement dynamics

Promotions accumulated at almost the same rate in all three conditions: 5.6, 5.7 and 5.4 per mission for C6, C6f and C6r, with audit pass rates of 22.4%, 22.9% and 21.5% (Figure 10). Promotions were front-loaded: 46% of C6 promotions occurred in the first eight rounds and 31% in the second half. Total evaluations per mission differed by less than 0.4% between conditions (about 364,000), confirming that the trial design was budget-neutral.

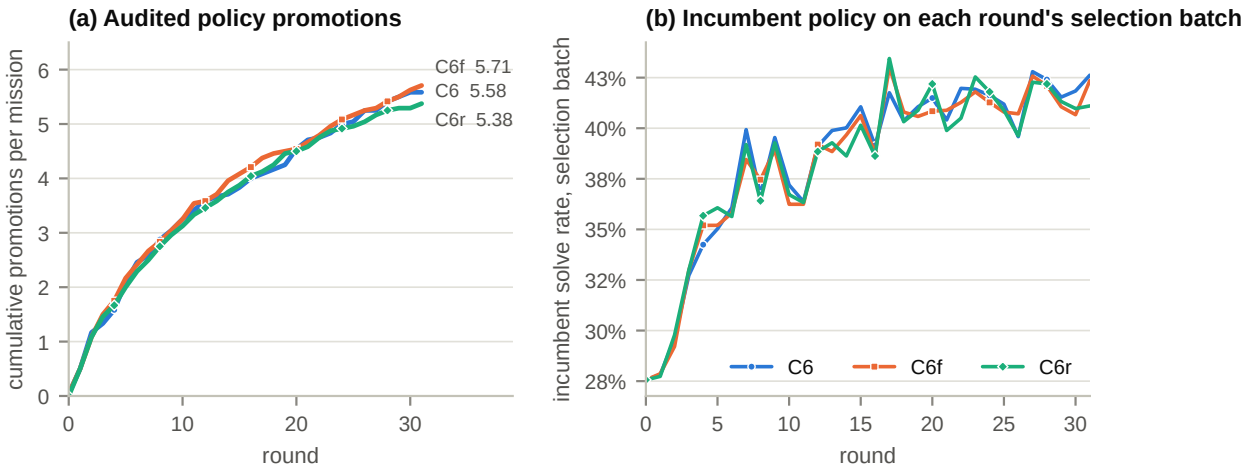


Figure 10: (a) Cumulative audited promotions per mission. (b) Solve rate of the incumbent policy on each round's selection batch (training families), averaged over seeds. The three conditions overlap throughout.

12.4 Exploratory: rewriting and benefit

Within C6, missions that adopted more Imp edits tended to finish slightly further ahead of their C6f twins (Spearman $\rho = 0.27$, $n = 24$; Figure 11). The association is weak, was not planned, and is not tested. We report it only because it bears on the direction of future work.

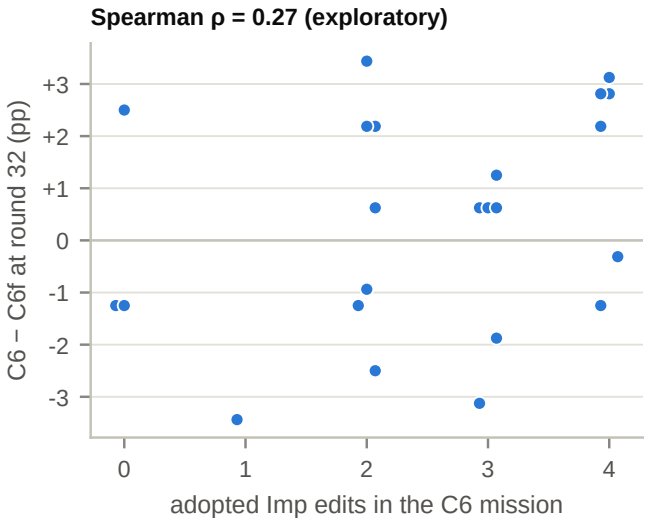


Figure 11: Adopted Imp edits per C6 mission against the round-32 difference from the same seed under C6f. Exploratory.

13 Results V: Autonomous Operation

All 72 confirmatory missions ran under the supervisor from start to finish. None required or received an intervention, none was restarted, and the watchdog never fired. Every mission ended in the completed state with a verified 32-round receipt chain (Table 15). The confirmatory run took 3 h 54 min of wall-clock time and 7.7 hours of summed mission time.

Table 15: Autonomy outcomes.

Criterion	Result	
Auto-1	72 missions: 0 interventions, 0 restarts, 0 watchdog kills; 72/72 completed with verified receipt chains	**PASS**
Auto-2	Frozen build: 14 of 14 groups pass, with faults injected during lmp trials	**PASS**
Auto-3	8 of 8 mutant supervisors detected (each fails 1–6 groups)	**PASS**

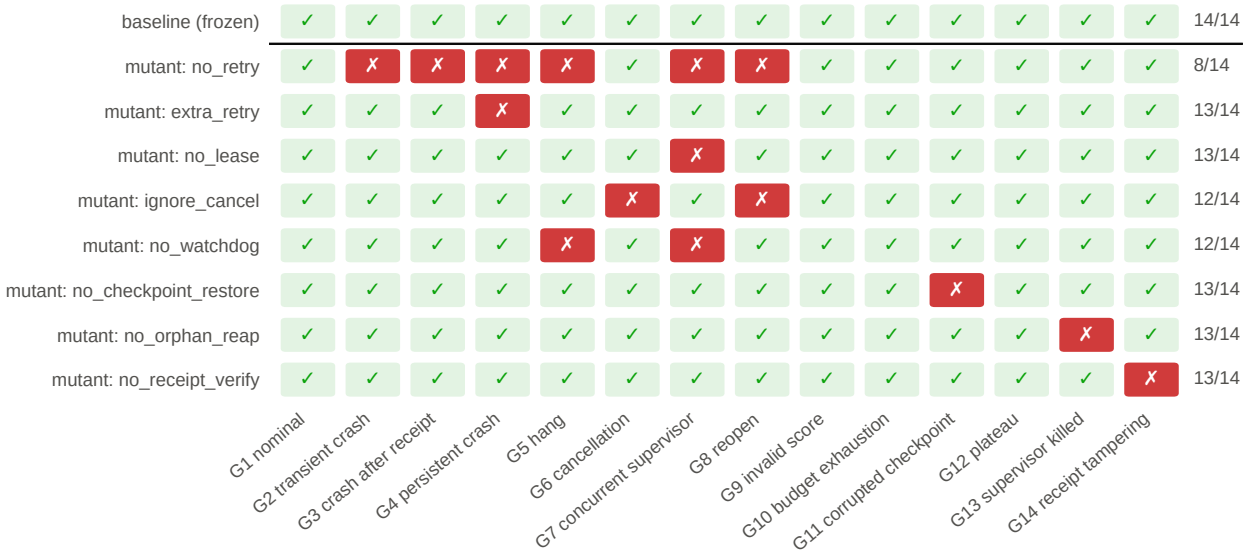


Figure 12: Fault-scenario suite on the frozen supervisor (top row) and on eight mutants, each with one safety mechanism removed. Green: the group's checks pass. Red: at least one check fails. Every mutant fails at least one group, including the group built to exercise the mechanism it removes.

The pattern of failures is specific. Removing crash retry breaks six groups, including all three crash groups. A supervisor that retries once too often fails only the persistent-crash group. Removing the lease lets a second supervisor run the same mission. Ignoring cancellation breaks cancellation and reopening. Removing the watchdog leaves hangs unrecovered. Disabling checkpoint restoration, orphan reaping or receipt verification each breaks exactly the group built for it.

14 Follow-up Studies: Is the Recursion Productive?

The confirmatory result left one question open: can rewriting Imp make the system a better improver at all? After the confirmatory run we carried out four further studies of that question (Table 16). Each had its own protocol, registered before its code was written, its own freeze record and gates, and seeds never used before. The first asks whether the mechanisms evolved in the confirmatory run are better improvers when transplanted into fresh missions. The other three redesign how a variant of Imp is validated. All four keep the task environment, the audit gate, the supervisor and the statistics of FELIX-C3. None of them changes the Tier A or Tier B results above, which stand as registered.

Table 16: The four follow-up studies. Each primary contrast compares round-32 test solve rate, paired by seed, with the same minimum effect of +1.0 pp used for Tier B.

Study	Design	Seeds	Outcome
Transplant test	Each evolved Imp from the confirmatory run is frozen into a fresh mission and compared with the default Imp and with a randomly drifted Imp. Primary: evolved − default, $p < 0.05$.	7201–7224 ($n = 24$)	**FAIL**: +0.49 pp, $p = 0.17$
Within-mission validation	A variant runs in a forked copy of the system for a four-round window and is judged on fresh tasks. Primary: rewriting − frozen, $p < 0.01$.	pilot 3301–3308 ($n = 8$)	NO-GO at Gate 1: pilot −1.04 pp, power 0.00
Cross-mission validation	A variant is judged over four whole missions on fresh seeds before it is adopted. Primary: evolved − default, $p < 0.01$.	pilot 3401–3408 ($n = 8$)	NO-GO at Gate 1: pilot +0.70 pp, power 0.15
Noise-aware cross-mission validation	As above, but a variant that passes a four-seed screen must also win on eight fresh, never-reused seeds by a margin. Primary: evolved − default, $p < 0.01$.	7501–7524 ($n = 24$)	**FAIL**: −1.12 pp, $p = 0.97$

The first two redesigns stopped at their pilot gate, which runs a confirmatory experiment only if the pilot predicts at least an even chance of meeting the primary criterion. Their confirmatory seeds (7301–7324 and 7401–7424) remain unused, and their pilot estimates rest on eight seeds and are descriptive. The third redesign had no pilot power gate and ran its confirmatory experiment once.

14.1 Transplant test

Design. For each of the 24 confirmatory seeds, the final Imp of the C6 mission was installed, frozen, in a new mission on a new seed (confirmatory seed + 100) as condition TE. Two comparison conditions ran on the same seeds: the default Imp (T0) and the final Imp of the C6r mission from the same donor seed (TR), a mechanism changed by coin-flip adoption. Within a seed the three missions shared every random stream, so the starting Imp was the only difference. The primary criterion (T-1) was TE − T0 $\geq +1.0$ pp with $p < 0.05$; T-2 (TE − TR) was to be tested only if T-1 passed.

Validity. All checks passed. The sources matched the Stage 3 freeze. A determinism rerun was byte-identical. The three donors that never adopted an edit (so that TE equals T0) produced runs identical to the default, confirming that the transplant was the only difference. All 72 missions reconciled and ran with zero interventions.

Table 17: Transplant test (round-32 test solve rate unless stated; $n = 24$).

Contrast	Mean (pp)	95% CI	Seeds $+/-/0$	p
T-1: TE − T0 (primary)	+0.49	[−0.22, +1.30]	11/8/5	0.17
T-2: TE − TR (not tested)	+0.35	[−0.38, +1.11]	13/10/1	0.24
TR − T0	+0.14	[−0.49, +0.79]	11/11/2	0.37
TE − T0, round 16	−0.89	[−1.76, −0.09]	8/10/6	0.94
TE − T0, held-out families	+0.31	[−1.15, +1.67]	11/9/4	0.26
TE − T0, development families	+0.56	[−0.30, +1.48]	11/10/3	0.16

T-1 **FAIL**. The evolved mechanisms were behind at the midpoint and slightly ahead at the end.

14.2 Is there headroom in the mechanism?

Before redesigning the rewriter we asked whether Imp can matter at all. On four development seeds, missions ran with Imp held for all 32 rounds at an alternative setting, paired with the default under identical random streams (Table 18). This scan was exploratory and not a criterion. The budget-split settings were configuration changes, since in FELIX-C3 the split is fixed; the next redesign made it part of Imp.

Table 18: Headroom scan: alternative settings held for whole missions, difference from the default at round 32 (pp; development seeds, $n = 4$).

Setting (parameter group)	Per seed	Mean
Faster vocabulary growth (generation)	+1.6, +4.4, +0.3, +1.9	+2.03
Budget split toward fewer candidates on larger batches (budget)	+4.7, +3.4, +0.3, 0.0	+2.11
Larger revisions (proposal)	+1.9, +0.6, +0.9, +3.1	+1.64
Stronger preference for one revision kind (proposal)	+0.9, −0.6, +0.3, +4.4	+1.25
Screening penalty (screening)	+1.9, +0.9, +1.3, −2.8	+0.31
Budget split toward more candidates on smaller batches (budget)	−3.4, +2.2, 0.0, +1.9	+0.16
Smaller revisions (proposal)	−1.9, −0.6, −0.9, +2.2	−0.31
No vocabulary growth (generation)	−2.8, −7.2, −3.7, −1.6	−3.83

The mechanism has real headroom: three settings were worth about +1.6 to +2.1 pp and one cost nearly 4 pp. But these are effects of holding a setting for a whole mission. Over the few rounds of a trial they amount to a few tenths of a point, while a single promotion moves a mission by several points. The pooled per-seed standard deviation of the paired differences was 2.1 pp.

14.3 Redesign 1: validation within a mission

Changes. Imp gained a twelfth parameter, the budget split: how the fixed per-round budget is divided between the number of candidate revisions and the size of the selection batch (4×192, 8×96 or 16×48). The generation parameters received wider bounds. The paired trial was replaced by *fork-and-validate*. At the start of each trial window the system copies its whole operational state into a main branch, which runs under the incumbent Imp, and a shadow branch, which runs under the variant. Both branches run the window on identical task batches, each with its own audited promotions. At the end, both are evaluated on a fresh batch of 300 training-family tasks, paired by task. If the shadow branch solves more tasks than the main branch by more than the margin, the system adopts the shadow branch's state and its Imp.

Conditions. C7 is the live system. In C7f the shadow branch runs the same Imp with different proposal randomness, so forking and branch selection are identical but Imp never changes. In C7v the shadow branch runs a variant of the default Imp and Imp reverts after every window. All arms share random streams and spend the same budget (evaluations within 0.4%).

Gates. Gate 0 passed: a determinism rerun was identical, all four trajectories were distinct, and headroom was 0.385. The fault suite passed 14 of 14 groups, with faults injected while both branches were live, and detected 8 of 8 broken supervisors. At Gate 1, 7 of 8 pilot missions adopted an Imp edit (6 required), but bootstrap power for the primary contrast was 0.00. The verdict was NO-GO.

Table 19: Within-mission validation: Gate 1 pilot (development families, $n = 8$).

Contrast	Mean (pp)	95% CI	Seeds $+/-/0$	p (descr.)
C7 − C7f (primary)	−1.04	[−2.81, +0.73]	3/5/0	0.86
C7 − C7v	−1.51	[−2.97, −0.16]	2/4/2	0.95
C7v − C7f	+0.47	[−1.35, +2.34]	4/3/1	0.34
C7 − C7f, round 16	−0.73	[−1.82, +0.36]	2/5/1	0.90

Why it failed. 67 of the 166 validations (40%) ended in an exact tie: neither branch had promoted anything that changed its validation result, so the comparison carried no information. Where the branches did differ, the difference reflected which branch happened to land a promotion, not the quality of the variant. Selection on such outcomes amounts to drift. Trying variants without keeping them was harmless (C7v − C7f, +0.47 pp), but keeping them was worse than reverting them (C7 − C7v, −1.51 pp).

14.4 Redesign 2: validation across missions

Design. The mission code of Redesign 1 was reused byte-identically, with forking disabled, so that a mission runs under one fixed Imp. Rewriting moved to a *campaign*:

1. The incumbent Imp is evaluated on a panel of four fresh training seeds, one full 32-round mission per seed, on development families only.
2. Each generation, the incumbent proposes a variant using its own step size. A variant that changes only editing-procedure parameters cannot change any mission, so it is redrawn from the same stream.
3. The variant runs one mission on each panel seed, sharing random streams with the incumbent's mission on that seed.
4. The variant is adopted if its total of test tasks solved exceeds the incumbent's by more than the incumbent's own margin.
5. After an adoption the panel is replaced and the new incumbent is re-evaluated, so that an incumbent's score is never a selected one.

Two independent campaigns ran for 12 generations each. The test arms ran fresh 32-round missions on new seeds. E used the final Imp of its assigned campaign and D the default. R was a zero-cost control: it replays the same proposal procedure but adopts exactly as many edits as E, at randomly drawn generations, regardless of evidence.

Gates and campaigns. Gate 0 passed. A determinism rerun was identical, and a campaign whose driver was killed mid-generation (twice, at different points) finished identically to an uninterrupted one. Headroom was 0.406, and the fault suite again passed 14/14 groups and detected 8/8 broken supervisors. The campaigns ran once:

- Campaign 0 adopted 5 edits. It moved the budget split toward fewer candidates on larger batches, sampled the vocabulary more widely, added one more vocabulary element per round, made revisions larger and shifted its regularisation preference. Three of these directions match settings the headroom scan favoured.
- Campaign 1 adopted 1 edit: wider vocabulary sampling, together with an increase of its own adoption margin.

All 124 campaign missions reconciled with zero interventions, and every adoption decision was replayed exactly from the stored mission records.

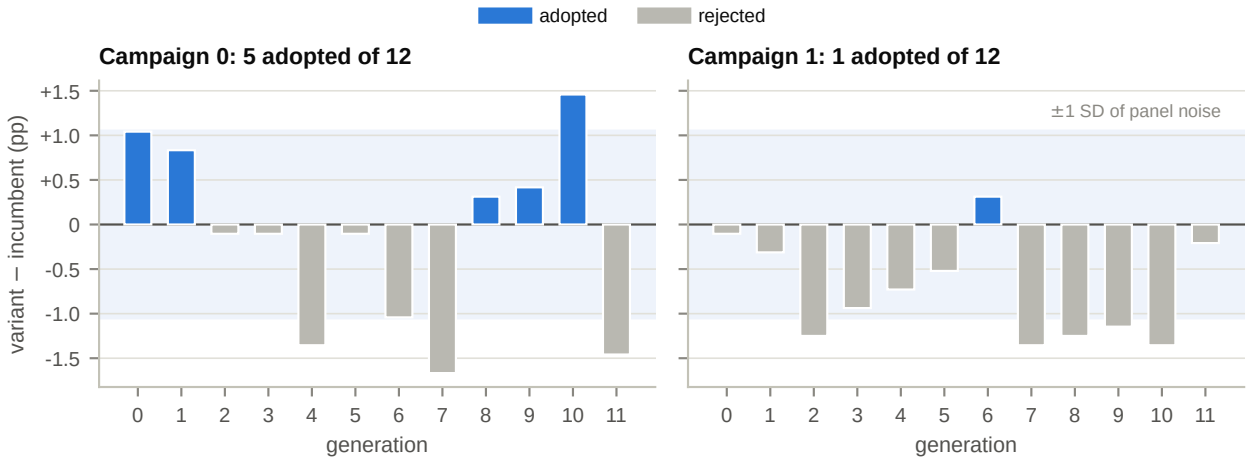


Figure 13: Cross-mission campaigns: variant minus incumbent per generation, expressed in points of test solve rate over the four-seed panel. The shaded band is about ±1 standard deviation of panel noise (≈ ±1.0 pp, estimated from the headroom scan). Several adopted edits fall well inside it.

Table 20: Cross-mission validation: Gate 1 pilot (all families, $n = 8$; seeds alternate between the two campaigns).

Contrast	Mean (pp)	95% CI	Seeds +/−/0	p (descr.)
E − D (primary)	+0.70	[−0.74, +1.99]	5/2/1	0.25
E − R	+1.21	[−0.20, +2.50]	6/1/1	0.078
R − D	−0.51	[−1.48, +0.39]	2/4/2	0.86
E − D, round 16	+0.35	[−0.98, +1.91]	4/4/0	0.46
E − D, held-out families	+1.09	[−1.72, +4.06]	3/3/2	0.20
E − D, campaign 0 seeds ($n = 4$)	−0.23	[−2.34, +1.88]	2/2/0	–
E − D, campaign 1 seeds ($n = 4$)	+1.64	[+0.47, +2.89]	3/0/1	–

Outcome. Bootstrap power for the primary contrast was 0.149 against the required 0.50, so the verdict was NO-GO. Because the pilot mean (+0.70 pp) lies below the +1.0 pp minimum effect, more seeds would not have raised the power. This is nonetheless the first design in which evidence-driven rewriting beat both the default (5 seeds up, 2 down) and random drift of the same size (6 up, 1 down) in direction. Adoption was still noise-limited. A panel difference has a standard deviation of about 10 tasks, the default margin was zero, and campaign 0 adopted two edits worth only 3 and 4 tasks (Figure 13).

Interruption (disclosed). Early in the campaigns the compute host was reclaimed by its provider during an idle period, and all processes stopped while both campaigns were in generation 0. The operator relaunched the unchanged command. The campaign drivers resumed from their committed state and the two missions in flight resumed from their round checkpoints. Nothing was rerun or discarded, and the replay check above covers the whole campaign. The automatic counters record no restarts and no interventions; the relaunch itself was an operator action and is recorded as such.

14.5 Redesign 3: noise-aware validation across missions

Design. Redesign 2 adopted edits worth less than its panel's noise. Redesign 3 keeps its campaigns, mission code and arms, and changes only the adoption rule, in two stages:

1. *Screen.* A variant runs on the current four-seed panel and goes on only if its total exceeds the incumbent's.
2. *Confirm.* The variant and the incumbent each run on eight fresh seeds, never used before or after. The variant is adopted only if its total over those eight seeds exceeds the incumbent's by more than Imp's own margin times eight. The initial margin is one task per seed, about 0.6 standard deviations of the confirmation difference.

The decision uses only the fresh seeds, so it carries no winner's curse from the screen. The minimum effect was unchanged (+1.0 pp, $p < 0.01$), with no pilot gate: the confirmatory ran once, provided each campaign adopted at least one edit.

Campaigns. Gate 0 passed (determinism, crash-only restarts at three points, headroom 0.419, fault suite 14/14 with 8/8 broken supervisors detected). The two campaigns adopted 5 and 4 edits (Figure 14). Every adopted edit won its confirmation by 9 to 16 tasks against a bar of 8. All 328 campaign missions reconciled with zero interventions, and both campaigns replayed exactly. Campaign 1's final mechanism moved, among other changes, to the budget split that the headroom scan had favoured (Section 14.2).

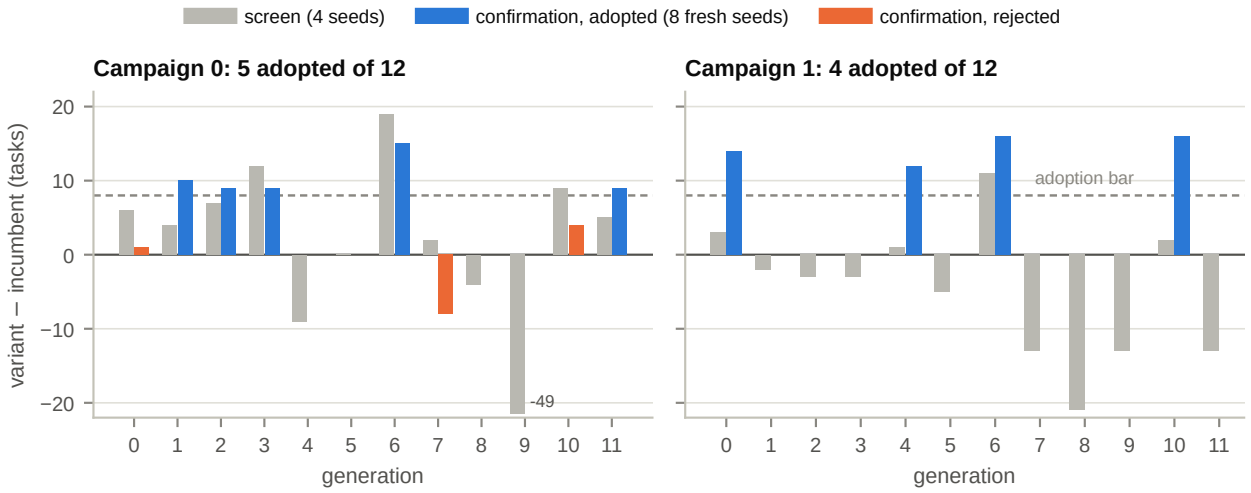


Figure 14: Noise-aware campaigns: per generation, the screen difference on four seeds (grey) and, for variants that passed it, the confirmation difference on eight fresh seeds (blue if adopted, orange if rejected). The dashed line is the adoption bar (8 tasks).

Confirmatory result. On 24 fresh seeds (7501–7524) the evolved mechanisms did *worse* than the default and than random drift (Table 21). Arm means at round 32 were 39.13% (default), 39.02% (random drift) and 38.01% (evolved). Second-half progress under the evolved mechanisms (+0.90 pp, $p = 0.0100$) also fell just

short of its bar, so the definitional tier did not pass either. The harm appears on the development families the campaigns trained on, not only on the held-out ones.

Table 21: Noise-aware validation, confirmatory ($n = 24$; seeds alternate between the two campaigns).

Contrast	Mean (pp)	95% CI	Seeds $+/-/0$	One-sided p
E − D (primary)	−1.12	[−2.16, −0.13]	8/15/1	0.97
E − R	−1.02	[−1.84, −0.17]	5/17/2	0.99
R − D	−0.10	[−1.20, +0.91]	11/11/2	0.45
E − D, held-out families	−1.56	[−3.33, +0.26]	8/15/1	0.95
E − D, development families	−0.97	[−2.12, +0.16]	9/15/0	0.94
E − D, campaign 0 seeds ($n = 12$)	−2.03	–	3/9/0	–
E − D, campaign 1 seeds ($n = 12$)	−0.21	–	5/6/1	–

Reading. Making validation strict removed the obvious explanation for the earlier nulls. Every adopted edit had won on eight seeds that played no part in proposing or screening it, yet the mechanisms built from those edits did not reproduce the gain on 24 further seeds. Two effects plausibly remain. With about six confirmations per campaign at a bar of 0.6 standard deviations, a neutral edit is occasionally adopted, and adopted edits compound. And an edit validated against the incumbent of the moment may interact badly with later ones. The design cannot separate the two. Either way, the parameters that Imp controls are worth too little, about two points at best (Section 14.2), for evidence gathered at this scale to find them reliably.

14.6 Does the improvement transfer to real bugs?

Every measurement in this paper is taken inside the synthetic domain the system trains on. A companion preregistered study [30] asks whether the improvement carries over to real bugs. It takes the 72 confirmatory end states of Section 10, unchanged and verified against the audit manifest, and runs them through the same frozen repair search on 164 bugs from QuixBugs and HumanEvalFix. The only new code is a sandboxed executor, which reproduced the original search exactly on 1,200 synthetic searches.

Under the identical budget, the C6 end states solved 25.81 real bugs against 20.96 for the system's own starting policy: +4.85 bugs [3.80, 5.84], with 23 of 24 seeds improved ($p = 3.0 \times 10^{-7}$). That is a relative gain of 23%, against 47% on the synthetic domain. Several checks support the result:

- the gain held at four times the training budget;
- it vanished when the learned parameters were permuted;
- it came entirely from HumanEvalFix, with none on QuixBugs.

C6, C6f and C6r transferred equally (−0.67 to +0.26 bugs between them). The same pattern held on the 72 end states of Redesign 3: about +5 bugs over the starting policy in every arm, and no difference between evolved and default mechanisms. The improvement loop therefore produces real repair competence. Rewriting its mechanism adds nothing to it, on synthetic tasks or on real ones.

14.7 Synthesis

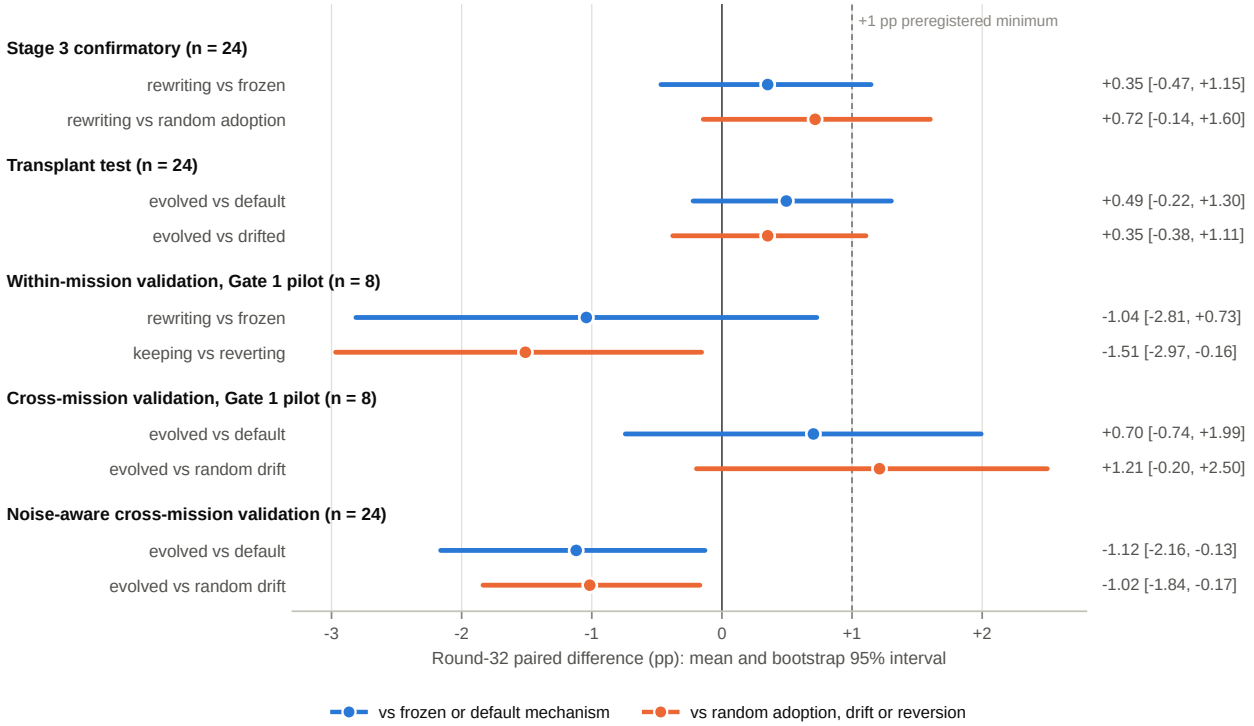


Figure 15: All causal contrasts across the five preregistered studies. Blue rows compare against a frozen or default mechanism and orange rows against random adoption, drift or reversion. Stage 3, the transplant test and the noise-aware redesign are confirmatory ($n = 24$); the first two redesigns are Gate 1 pilots ($n = 8$). None meets its preregistered criterion.

Five preregistered tests of productive recursion, and none passes (Figure 15). The estimates are slightly positive in the confirmatory run and the transplant test, and negative when variants are judged inside a single trajectory. They are positive in the pilot that judged variants across whole missions, and negative again once that design was made strict and run at full size. The one positive-looking design did not survive its own confirmation. If productive recursion exists in this system, it is smaller than the noise that evidence at this scale can resolve. Meanwhile the base improvement loop gains roughly +12 pp in every mission, and that gain carries over to real bugs (Section 14.6).

15 Discussion

15.1 What the result establishes

Under a protocol fixed before any code was written, FELIX-C3 satisfies every clause of Liu et al.'s Level-4 definition as we operationalised it. It proposes, validates and applies modifications without a human. It rewrites its own improvement mechanism, adopting only variants that win a paired, budget-neutral trial, and in most missions it rewrites the very procedure that does the rewriting. That last step is admitted by the same trial but is not measured by it, a limitation we return to below. Its test performance keeps improving through the second half of long missions in a bounded domain. It did all of this in 72 missions with no operator, under a supervisor whose fault suite was itself shown to be able to fail. The earlier stages of the programme, despite passing their own tests, did not meet this bar, because their improvement mechanism was fixed.

The granularity of rewriting differs from the systems Liu et al. place at Level 4. The Darwin Gödel Machine and its successors rewrite agent code; FELIX-C3 rewrites the values of an explicit, schema-bound mechanism, including the parameters of its own editing procedure. The definition does not specify a granularity, and we do not claim equivalence of capability, only that the definition's clauses are met.

15.2 Why the recursion was not productive

The causal criteria failed, and the gate record had said they would. Several observations point to why.

- *Headroom visible only over whole missions.* In development, the best fixed alternative setting of lmp we probed changed final solve rates by at most a few points. A later scan found settings worth about two points when held for a whole mission (Section 14.2), but a trial lasts only a few rounds.
- *A weak trial signal.* Each trial compares four candidates per side on a 96-task batch over a few rounds. Evidence-based and coin-flip adoption ended up adopting edits at similar rates for most parameter groups. Edits to the screening parameter and to the editing procedure are not exercised during their own trial at all, so for 13 of the 60 adoptions the trial outcome could not reflect the edit.
- *A strong base loop.* All three conditions gain about 10.6 pp in the first half and 1–2 pp in the second, whatever lmp does. The improvement loop, not its mechanism, carries the progress.

Trials did not appear to cost anything: variant-side candidates won half of the audits and promotions during trial windows, and total evaluation budgets were equal. The null is therefore a null of benefit, not evidence of harm.

The follow-up studies (Section 14) sharpen this diagnosis. Validating rewrites inside one trajectory selects luck, because a single promotion outweighs the few-round effect of any setting. Validating them across whole missions pointed the right way in a pilot. When that design was made noise-aware and run at full size, however, the adopted edits did not generalise. What limits the recursion is therefore not chiefly how rewrites are validated. It is how little the rewritable parameters are worth.

15.3 Implications for Level-4 classification

Our result shows that a system can meet the Level-4 definition while its self-referential rewriting contributes nothing measurable to its progress. Taxonomies that grade systems by which roles they perform will classify such a system at Level 4, correctly by the letter of the definition. We suggest that a Level-4 claim should, in addition, report two counterfactuals at matched budget: the same system with its improvement mechanism frozen, and the same system with mechanism changes adopted without evidence. Without them, "rewrites its own improvement mechanism" cannot be distinguished from rewriting for its own sake. This is the same distinction Chen et al. draw between self-refinement against a fixed evaluator and recursion that actually

closes the loop [10]. In FELIX-C3 the policy-level validator is external and fixed, and it is robust; the mechanism-level validator is internal, and that is where the signal is weakest. The follow-up studies add four more counterfactual tests of this kind, and none supports productive recursion. This is why we report the Level-4 classification and the causal results side by side.

15.4 Next steps

The transplant test, three redesigns of the rewriter and an external-benchmark transfer test have since been run (Section 14). Two directions remain. First, give the mechanism more room to matter: structural edits to `lmp` rather than parameter edits, or a stronger proposer of edits. A study in which a language model proposes the rewrites, under the same validator, has been designed and will be reported separately. Second, a second task domain is required before any question of Level 5, which demands that the improvement capability transfer across domains.

16 Limitations and Threats to Validity

Operationalisation. The criteria are our reading of a definition written to classify architectures. Other readings are possible: "long-term" could demand far longer horizons than 32 rounds, and "rewrite its own improvement mechanism" could be read as requiring code-level rewriting. We fixed our reading in advance and report it in full (Table 1).

The standard itself is unreviewed. Liu et al. and Chen et al. are 2026 preprints that had not been peer reviewed when this study was registered. If the published definitions change, the classification may change with them.

Scope of rewriting. Imp has a fixed schema of 11 parameters. The system can change their values, including the values that govern how it changes them, but it cannot add parameters or change the schema.

Unmeasured procedure edits. Changes to the editing procedure are admitted by trials that cannot measure their effect (Section 11.5). Self-reference is therefore established structurally, and the procedure is demonstrably rewritten by itself, but the benefit of any individual change to the procedure is not validated.

Low power for Tier B. The study was powered for Tier A, not Tier B. The B-1 interval, $[-0.47, +1.15]$ pp, excludes large benefits of Imp rewriting but not small ones.

Follow-up pilots. The first two redesigns stopped at Gate 1 on eight pilot seeds each. Their estimates are descriptive and their intervals wide, and the per-campaign figures for the cross-mission design rest on four seeds each. The headroom scan used four development seeds and was exploratory.

Domain and level. The domain is small and synthetic, and absolute performance is modest (about 40% of test tasks solved). The companion transfer study shows partial carry-over to real bugs, but only for the fault types nearest the synthetic domain, and with a one-step edit set that can reach at most a third of the real bugs. Results may not carry over to open-ended domains or to systems built on large models.

Test-task overlap. In the development families, many test tasks share program text with some training task while carrying different examples; in a check of all 24 confirmatory seeds, between 162 and 184 of the 240 development-family test tasks per seed did (73% on average), and none was identical. The held-out families never appear in training, and none of their test tasks shares program text with a training task. The overlap is identical across conditions and cannot favour any of them, but it inflates absolute solve rates on development families.

Development and hold-out. Development missions used all 16 families, because the held-out set was determined only at the freeze, from the freeze hash. Development could therefore not target the held-out families specifically, but they were not unseen during development.

Registration and attestation. The protocol and the freeze, gate and results records were posted to a hosted document store with server-assigned timestamps, not to a public registry. There is a single implementation, the evidence is self-attested, and there has been no independent replication. The study was carried out with substantial AI assistance (Section 17.5). The audit package (Section 17) is intended to make such replication possible.

17 Integrity, Reproducibility and Disclosure

17.1 Commitments

Table 22 lists the hashes committed during the study; source-file hashes are in Appendix D. Sources were verified against the freeze before each gate, before the confirmatory run and after it. The confirmatory criteria were computed by the frozen runner directly from the mission records.

Table 22: Integrity commitments (SHA-256).

Artifact	SHA-256
Freeze hash (sources, configuration, initial lmp)	`550193d51cb08fc2971000fa49e09ac7f6e627ae14b74fa07ee8c484048b3801`
Freeze record file	`0b791ed0a3995197ddd09186cac5b5621eec166325dda44bbfc3a7d6c60394e0`
Gate 0 result	`5cc35485b7a4740981468705657915 97ccaa9861d0ef20ea2d053dd2428450c4`
Gate 1 result	`41b87945d09994578ceae7995056517a03c6537ee9146e52b3f22780879ea71e`
Autonomy suite result	`c0c7517140acb8a3b166dbcff443902ac17ae3e07d9f5951afc55e5609b8e6ac`
Confirmatory result	`7dc761ec60d6b384a42e9ba055759ea9a6f07836b80e57d3caf4cf6a43fa9c11`
Audit package (archive)	`596b7796f07e309450c1cad567209177a4b95147c8b3c1147eec85dea5023618`

The follow-up studies committed their own hashes, listed in Table 23. Each freeze record was posted before any of that study's gate or confirmatory missions ran.

Table 23: Follow-up commitments (SHA-256).

Artifact	SHA-256
Transplant: freeze hash	`ac2bd909ed964e9bde454dc21cb425e98fd6992f452d38cf3ef0d9d961327bb5`
Transplant: result	`169f9ea335456fe300682517a35b271a081841ca4e23d5beea05a10cd5c6b37c`
Within-mission: freeze hash	`4362e6fee9b6a1f0b7bfcd090b660f384d889adcb3c404a9497d978a09bc383d`
Within-mission: Gate 1 result	`870dd76606e21e22f68a345204ecd5e4788ddcb6357c92e9b4b1250780166cf4`
Cross-mission: freeze hash	`28bced02e2d7f1f697602a8816479e8b12d0638f3c44c5cdd3367ba0f87ca87f`
Cross-mission: campaign record	`195bff7363d32a420299b990b961091155275aa2278856d8bdb24bcc690cbef6`
Cross-mission: Gate 1 result	`985369d69bbe1fc7eed9b8c73e464203e68f87c8009e973a84cda5e6c31656d6`
Follow-up audit package (archive)	`c56d6fb7a1b36205cff57ba4d9ff1b9127819c83b79ce210358638353cda79b7`
Noise-aware: freeze hash	`6b1fb7968c5a2640d7db5d9a90a3f59f2d363d7fd2d5e8f1ea21f02f0479c5a5`
Noise-aware: campaign record	`655fb0595ba1696b9f3ec4557360983805a59f7174c82f4d38391b0ec67be5aa`
Noise-aware: Gate 1 result	`7b43fa5d00ab8897d90de9df7f93198bb011fb90934e61438eac04ea73cb4b47`
Noise-aware: confirmatory result	`8071d7fe76604e1355c1dcc7328df8e1e70e620bc80db48b3546e3358e0a5d6c`
Transfer study: freeze hash	`0b9617b8d2fd774dda7a6fec001fbb9d429888617ceb3730a4874a7fea37aec0`
Noise-aware audit package (archive)	`d899c71e1fb1a2f51a485e6ce69ddcaa7cd829ba14de9c0b49160c9312b2ab85`
Transfer study: result	`f1c95ca742edbb63bfc54e7f9fbf5452b497aefa04caa399f4dfe36cbcf9c289`
Transfer audit package (archive)	`8aaad34a97ab414f39ccdc8ffa188bd1f0b963f89c5e8cebf8b7bcd5f9a2d815`

17.2 Determinism

Gate 0 verified determinism on a development seed. After the confirmatory run, one confirmatory mission (C6, seed 7110) was rerun from scratch outside the study directory. Its receipt chain and its final state were byte-identical to the originals, including the full lmp version history.

17.3 Deviations and corrections

Launch before the freeze record. Gate 0 and the fault suite were launched at 02:44:39, two seconds after the freeze and 27 seconds before the freeze record was posted. No result of either existed when the record was posted; the first result appeared at 03:15. Gate 0 uses only development seeds, and the pilot and confirmatory seeds were untouched. We regard this as a procedural slip without effect on the evidence, and we disclosed it in the freeze record itself.

A descriptive error in the gate record. The gate record states that C6 finished below C6f on five of the eight Gate 0 seeds. Recomputing from the Gate 0 result file gives six of eight (Figure 3a). The error is in a descriptive note, not in any gate check, and this paper uses the corrected count.

Follow-up studies. Three further items are disclosed in the follow-up records.

- The first smoke test of the transplant pipeline used three seeds from its confirmatory range with a tiny configuration. Only completion status was seen; the run was deleted unopened, the smoke-test mode was moved to development seeds, and the slip was disclosed in the freeze record.
- Before the within-mission freeze, the task environment was made about twice as fast. The change was verified output-identical on 46,549 generated variants from 2,400 programs, and on a full-size four-round mission whose records were byte-identical. It is carried byte-for-byte into the cross-mission build.
- The cross-mission campaigns were relaunched once by the operator after the host was reclaimed (Section 14.4).
- The noise-aware campaigns shared their two-processor host with the transfer study and with development runs of another study for part of their duration. Missions are budgeted in evaluations, not time, so this affected only wall-clock time. No mission hit the hang watchdog.

Everything else as registered. No source, rule, seed or threshold changed after the freeze. No seed was excluded, and the confirmatory experiment ran once. Design changes made before the freeze, on development seeds, are listed in Appendix B.

17.4 Attempt accounting

This is attempt 1 under protocol v3, and it is the only confirmatory attempt under that protocol. Stages 1 and 2 were separate studies with separate protocols, seeds and single confirmatory runs. Earlier, separately built versions of the runtime module reported Level-4 results that did not survive audit; they are not counted here and are not used as evidence. Each follow-up study is attempt 1 under its own protocol. The transplant test ran its confirmatory experiment once. The first two redesigns stopped at Gate 1, so they made no confirmatory attempt, and their confirmatory seeds remain unused. The noise-aware redesign ran its campaigns once and its confirmatory experiment once. The transfer study ran its confirmatory evaluation once.

17.5 Author contributions and use of AI assistance

The research programme and its direction, the decision to test the system against Liu et al.'s definition, and the decisions to proceed, to publish and to disclose are the author's. Study design and protocols, the implementation of the system and of the experimental harness, the execution and monitoring of all runs, the statistical analysis, the figures and the drafting of this manuscript were carried out with substantial assistance from an AI system (Claude, Anthropic), working under the author's direction in a sandboxed computing environment.

The AI system was not part of the system under test. It did not take part in any mission, campaign or improvement cycle: every self-modification reported here was proposed, validated and applied by the system itself, and every mission ran under the supervisor with zero interventions. All code, records and analyses are retained in the audit packages, so that each step can be checked independently of who or what performed it. Versions 1.0 and 1.1 of this paper did not include this statement; it was added in version 1.2. The same applies to the follow-up studies added in version 1.3 and to the companion transfer study.

17.6 Audit package and reproduction

The frozen sources, the freeze record, every mission record of the gates, the fault suite and the confirmatory run (states, receipt chains, supervisor event logs) and the result files are held in an audit package with a per-file SHA-256 manifest. Because the package discloses the architecture, it can be made available to reviewers under a confidentiality agreement. With it, an auditor can:

(i) verify the source hashes against the freeze record;

(ii) recompute every Tier A, Tier B and autonomy criterion from the stored mission records and compare with the confirmatory result file;

(iii) walk each mission's receipt chain and `lmp` hash chain;

(iv) rerun any single mission from its specification and compare receipts;

(v) rerun the fault suite and the mutation analysis.

A second package holds the follow-up studies: their frozen sources, freeze records, gate, campaign and transplant mission records, and result files, with its own per-file manifest. Third and fourth packages hold the noise-aware redesign and the transfer study. The same confidentiality terms apply.

18 Conclusion

We set out to test a self-modifying system against a published definition of Level-4 bounded recursive self-improvement, under conditions that could fail. The earlier stages of the programme kept their improvement mechanism fixed and so fell short of the definition. The third stage made the mechanism an explicit object that the system rewrites, including the procedure that rewrites it. In a preregistered, single-attempt experiment on 24 fresh seeds, it met every clause of the definition and ran without an operator under a supervisor that passed a fault suite shown to be able to fail.

The same experiment shows the limit of that achievement. Rewriting the improvement mechanism did not make the system measurably better than leaving the mechanism alone or rewriting it at random. Meeting the definition and benefiting from recursion are different claims, and only the first is supported here. Four follow-up studies confirm the distinction. Evolved mechanisms were not better improvers when transplanted, and judging rewrites inside a trajectory made matters worse. Judging them across whole missions pointed the right way in a pilot. Made strict and run at full size, the same design produced mechanisms slightly worse than the default. The progress the system does make is real: a companion study finds that the policies it produces repair more real bugs than its starting policy. That progress comes from the improvement loop, not from the recursion. We think Level-4 claims, including ours, should report both the definition and the counterfactual, and, where possible, transfer beyond the training domain. The next experiments are a stronger or structural proposer of mechanism edits, a second domain, and independent replication.

References

[1] Irving John Good. Speculations concerning the first ultraintelligent machine. *Advances in Computers*, 6:31–88, 1966. doi: 10.1016/S0065-2458(08)60418-0.

[2] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. Technical Report IDSIA-19-03, arXiv:cs/0309048, 2003.

[3] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-Taught Optimizer (STOP): Recursively self-improving code generation. In *Proceedings of the First Conference on Language Modeling (COLM)*, 2024. arXiv:2310.02304.

[4] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A self-referential agent framework for recursive self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2410.04444.

[5] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2408.08435.

[6] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.22954.

[7] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. arXiv:2504.15228, 2025.

[8] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel Machine: Human-level coding agent development by an approximation of the optimal self-improving machine. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.21614.

[9] Shuaiqi Liu, Zhengkai Lin, Yuxiang Zhang, Yuanyi Ren, Yue Wu, Yongbin Li, Zheng Wang, Zhihang Fu, and Jieping Ye. The path to recursive self-improving agents: Foundation, framework, and future directions. Preprints.org, preprint 202608.0051, version 1, 2026. Not peer reviewed.

[10] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops. arXiv:2607.07663, 2026.

[11] Roman V. Yampolskiy. From seed AI to technological singularity via recursively self-improving software. arXiv:1502.06512, 2015.

[12] Marcin Andrychowicz, Misha Denil, Sergio Gómez Colmenarejo, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016. arXiv:1606.04474.

[13] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, 2020. arXiv:2003.03384.

[14] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 30(3):286–297, 2000. doi: 10.1109/3468.844354.

[15] SAE International. Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles. Technical Report J3016_202104, SAE International, 2021.

[16] Meredith Ringel Morris, Jascha Sohl-Dickstein, Noah Fiedel, Tris Warkentin, Allan Dafoe, Aleksandra Faust, Clement Farabet, and Shane Legg. Position: Levels of AGI for operationalizing progress on the path to AGI. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, 2024. arXiv:2311.02462.

[17] K. J. Kevin Feng, David W. McDonald, and Amy X. Zhang. Levels of autonomy for AI agents. arXiv:2506.12469, 2025.

[18] George Candea and Armando Fox. Crash-only software. In *Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*. USENIX Association, 2003.

[19] Jim Gray. Why do computers stop and what can be done about it? Technical Report 85.7, Tandem Computers, 1985.

[20] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi: 10.1109/C-M.1978.218136.

[21] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

[22] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

[23] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. arXiv:1709.06560.

[24] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[25] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.

[26] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

[27] Joseph L. Hodges and Erich L. Lehmann. Estimates of location based on rank tests. *The Annals of Mathematical Statistics*, 34(2):598–611, 1963. doi: 10.1214/aoms/1177704172.

[28] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

[29] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi: 10.1145/3318162.

[30] Junai P. Felix. Do self-improved repair policies transfer to real bugs? a preregistered external-benchmark test of a level-4 bounded self-improving system. Companion preprint, version 1.1, 2026.

A Per-Seed Confirmatory Results

Table 24: Test solve rate (%) per confirmatory seed at rounds 0, 16 and 32, and Imp activity in C6 (trials, adopted edits, adopted edits that changed the editing procedure). Round 0 is identical across conditions.

		C6		C6f		C6r		C6 Imp		
Seed	R0	R16	R32	R16	R32	R16	R32	trials	adopted	meta
7101	25.3	40.3	40.9	39.1	38.8	37.2	39.7	7	4	2
7102	27.8	35.0	37.8	35.3	35.0	37.5	37.2	7	4	2
7103	26.9	31.9	38.4	34.1	37.8	36.9	37.8	7	3	1
7104	23.4	38.4	38.4	37.8	39.7	40.3	41.2	7	0	0
7105	25.6	36.9	42.5	35.9	39.1	39.7	40.0	6	2	1
7106	27.5	38.8	40.0	35.3	41.9	38.8	38.1	12	3	1
7107	29.1	38.8	41.2	38.1	40.6	39.1	40.6	5	3	1
7108	26.2	38.4	40.9	41.6	41.9	40.3	40.9	7	2	0
7109	29.7	39.1	39.1	40.3	41.6	39.1	40.6	5	2	1
7110	25.3	37.2	39.1	35.3	36.2	30.0	34.1	7	4	2
7111	25.0	33.8	34.1	32.8	33.4	35.3	34.4	7	3	2
7112	28.4	36.9	38.8	37.8	36.6	37.5	36.2	7	2	1
7113	29.1	41.2	40.9	37.5	42.2	41.6	44.4	7	2	0
7114	24.4	34.4	36.2	35.9	37.5	37.2	36.6	7	0	0
7115	29.4	41.6	41.2	40.6	40.6	37.8	40.9	4	2	1
7116	25.9	38.8	38.8	40.3	41.9	37.5	38.4	7	3	2
7117	27.8	40.9	42.8	36.9	39.7	40.3	40.6	11	4	3
7118	30.9	39.7	42.8	41.6	41.6	40.9	45.6	7	3	1
7119	27.2	35.3	35.3	37.8	38.8	33.4	35.9	7	1	0
7120	25.9	40.6	41.6	38.4	39.4	39.4	38.4	7	2	1
7121	29.4	35.6	42.2	40.3	42.5	40.6	41.9	7	4	0
7122	29.4	39.4	43.8	42.8	45.0	43.1	45.0	5	4	2
7123	28.4	37.8	41.9	36.2	39.4	36.9	37.5	7	0	0
7124	28.7	40.9	43.8	39.1	43.1	38.4	39.1	6	3	1
Mean	27.4	38.0	40.1	38.0	39.8	38.3	39.4	6.9	2.5	1.0

B Development-Phase Disclosure

All runs below were made before the freeze, on development seeds and all 16 families, and were disclosed in the freeze record before any gate ran. They are not evidence for any criterion.

Table 25: Development runs. Test solve rate at the start, midpoint and end of each mission.

Design	Seed	Condition	Trajectory	Adopted / trials	Note
First (40 rounds)	1302	C6	0.259 → 0.362 → 0.384	0 / 9	variant could use incumbent's additions
	1302	C6f	0.259 → 0.359 → 0.412	–	
	1303	C6	0.247 → 0.362 → 0.384	1 / 9	the one adoption changed the editing procedure
	1303	C6f	0.247 → 0.322 → 0.375	–	
Fixed-Imp probe	1302	alternative A	0.259 → 0.394 → 0.412	–	frozen Imp with one generation setting changed
(40 rounds)	1303	alternative A	0.247 → 0.394 → 0.419	–	
	1302	alternative B	0.259 → 0.366 → 0.394	–	a different generation setting; no gain
	1303	alternative B	0.247 → 0.334 → 0.372	–	
Final (32 rounds)	1304	C6	0.216 → 0.281 → 0.341	2 / 7	isolated trials; finer trial statistic
	1304	C6f	0.216 → 0.347 → 0.362	–	
	1305	C6	0.231 → 0.381 → 0.400	2 / 8	both adoptions changed the editing procedure
	1305	C6f	0.231 → 0.356 → 0.372	–	

Known weaknesses stated before any gate. The freeze record listed three: (1) Tier B was likely to fail, since the mean development difference between C6 and C6f was about zero; (2) sustained progress also occurred with a frozen Imp in development, so A-4 would show that the overall process sustains progress, not that Imp rewriting causes it; and (3) rewriting here is parameter-level rewriting of an explicit specification, not arbitrary code rewriting.

C Statistical Details

Exact Wilcoxon test. For paired differences $d_1, \ldots, d_n$ with zeros removed, let r_i be the midrank of $|d_i|$ and $W^+ = \sum_{d_i>0} r_i$. Under the null hypothesis each sign is positive with probability $1/2$ independently. Working with doubled ranks $2r_i$, which are integers, the null distribution of $2W^+$ is computed exactly by the recursion $c_k(s) = c_{k-1}(s) + c_{k-1}(s-2r_k)$, and the one-sided p-value is $\sum_{s \geq 2W^+_{\text{obs}}} c_n(s)/2^n$. The procedure is exact under ties, conditional on the observed ranks.

Gate 1 power. Power was estimated by drawing 10,000 samples of size 24, with replacement, from the eight pilot differences, and counting the fraction in which the criterion (minimum mean and p-value) would hold. Pilot differences for B-1 had mean +0.10 pp and both signs, giving power 0.0104. For A-4 all eight pilot values were positive, giving power 1.00; this estimate is optimistic by construction but was confirmed by the result.

Effect sizes. Table 26 gives standardised paired effects $d_z = \bar{d}/s_d$ for the main contrasts.

Table 26: Standardised paired effect sizes ($n = 24$).

Contrast	Mean (pp)	SD (pp)	d_z
C6 second-half progress (A-4)	+2.12	2.09	+1.01
C6 − C6f, round 32 (B-1)	+0.35	2.06	+0.17
C6 − C6r, round 32 (B-2)	+0.72	2.28	+0.31
C6 − C6f, held-out families	+0.89	5.40	+0.16
C6 − C6f, second-half progress	+0.33	2.44	+0.13

D Source Hashes

Table 27: SHA-256 of the eight frozen source files, listed by role. Three are carried over byte-for-byte from earlier stages.

File	SHA-256	Note
Task environment	`9ce16966d7b973ada54063b3a0631da2878b3e297a8775739d8ff3823d79ef49`	as Stage 2
Operators	`ef4107850d02f11827a330a0e5cff05bd5c500423821a329e07c0875d048e34b`	extended
Improvement mechanism	`4491b1ab7f25671dcdd4e901458ca86af05ff06e8e393088c6138879a11f4f06`	new in Stage 3
Mission loop	`d7b155d1f9bfdf24186989704b70429c0b845a951d9cc050f69c0c53518431ae`	extended
Supervisor	`46a9afd5ef2cdb8cfa847505770030faea9eacdce6c7e9ee6f8f7141464c401e`	as Stage 1
Fault suite	`c45fd0456dd362b9452e11f13465589ba47e832e0e0f0b508ce1dd6b4828595d`	extended
Runner and criteria	`bd6255db62188179701450cffb7e87d8dfa515fe83dfaed33cbcd87e9af3a8ad`	extended
Statistics	`ee8c288d9207f1ded2a896826c1a5da33e7f0c8c4033681f4753e90a3cb7d0c6`	as Stage 1